

The Chinese University of Hong Kong, Shenzhen
Personal study materials · Unofficial

CSC5041 Database Systems

English Study Notes

Weeks 1–5 · Chapters 01–06

Adapted from the supplied academic report template.

Original teaching materials remain unchanged.

LaTeX edition: 20 October 2026.

2026-10-20

Contents

01 Database Fundamentals: English Study Notes **5**

Scope and learning goals . . . 5
1. Four foundational concepts . . . 5
2. Why not manage everything with files? . . . 6
3. What does a DBMS provide? . . . 7
4. Data models, metadata, schemas, and instances . . . 7
5. Three stages of database design . . . 8
6. Database languages and users . . . 8
7. Three-schema architecture [supplement] . . . 10
8. Data independence [supplement] . . . 10
9. Understanding the DBMS component diagram . . . 11
10. Database types: recognize categories, not live rankings . . . 12
11. Common distinctions . . . 12
12. Revision checklist and short-answer language . . . 13
Sources and preparation record . . . 13

02 ER Modelling: English Study Notes **14**

Scope and learning goals . . . 14
1. What problem does ER modelling solve? . . . 14
2. Entities, entity types, and entity sets . . . 14
3. Attribute types: distinguish three dimensions . . . 15
4. Keys: check uniqueness and minimality separately . . . 16
5. Strong entities, weak entities, and partial keys . . . 17
6. Relationships, relationship attributes, and roles . . . 18
7. Cardinality constraints: ask about both maxima . . . 19
8. Participation constraints: ask about both minima . . . 19
9. Lecture notation reference . . . 20
10. Worked example: library requirements to an ER diagram . . . 20
11. A procedure for modelling written requirements . . . 22
12. Common distinctions . . . 22
13. Revision checklist and short-answer language . . . 23
Sources and preparation record . . . 23

03 Relational Model and Integrity Constraints: English Study Notes **24**

Scope and learning goals . . . 24
1. From the ER model to the relational model . . . 24
2. Schemas, instances, degree, and cardinality . . . 24
3. Basic properties of a relation . . . 25
4. Superkeys, candidate keys, and primary keys . . . 26
5. Foreign keys: direction, repetition, and composite references . . . 27
6. NULL and the limits of foreign-key rules . . . 28
7. What do the three integrity constraints check? . . . 28
8. Running example: schema, constraints, and initial state . . . 29
9. Insertion: check the new tuple and reference targets . . . 30
10. Deletion: follow references in the reverse direction . . . 30
11. Modification: consider both old and new values . . . 31
12. Common distinctions . . . 32
13. Revision checklist and short-answer language . . . 32
Sources and preparation record . . . 33

04 ER-to-Relational Mapping: English Study Notes 34

Scope and learning goals . . . 34
1. Mapping inputs, outputs, and notation . . . 34
2. Overview of the seven steps . . . 35
3. Step 1: strong entities and ordinary attributes . . . 36
4. Step 2: weak entities and identifying relationships . . . 36
5. Step 3: binary 1:1 relationships . . . 37
6. Step 4: binary 1:N relationships . . . 38
7. Step 5: binary M:N relationships . . . 38
8. Step 6: multivalued attributes . . . 39
9. Step 7: n-ary relationships and key selection . . . 40
10. Worked example: the lecture's company model . . . 41
11. Constraints to check after mapping . . . 42
12. A practical answer-writing procedure . . . 42
13. Common distinctions . . . 43
14. Revision checklist and short-answer language . . . 43
Sources and preparation record . . . 44

05 Relational Algebra: English Study Notes 45

Scope and learning goals . . . 45
1. Notation and closure . . . 45

2. Running example: specify the inputs first . . . 46
3. Selection σ: filter tuples, retain all attributes . . . 47
4. Projection π: retain attributes and eliminate duplicates . . . 47
5. Rename ρ: distinguish roles and conflicting names . . . 48
6. Union, intersection, and difference: align schemas first . . . 48
7. Products and joins: all pairings versus correct pairings . . . 49
8. Composing queries: preserve identity, project names last . . . 50
9. Division $\div$: express "every target is covered" . . . 50
10. Expanding division with difference, and empty targets . . . 51
11. γ: grouping and aggregation . . . 52
12. Equivalent rewrites: check the conditions first . . . 52
13. A procedure for translating English requirements . . . 53
14. Common distinctions, revision checklist, and short-answer language . . . 53
Sources and preparation record . . . 55

06 SQL Queries: English Study Notes **56**

Scope and learning goals . . . 56
1. Six common clauses: written and logical order . . . 56
2. Running example: add terms and missing scores to Chapter 05 . . . 57
3. Basic queries, aliases, strings, and DISTINCT . . . 58
4. NULL and three-valued logic . . . 59
5. Multiple-table joins: identify the full association . . . 59
6. Self-joins and ordering . . . 60
7. Aggregates: COUNT, SUM, AVG, and NULL . . . 61
8. GROUP BY and HAVING: one row per group . . . 61
9. Subqueries: scalar versus multiple rows . . . 62
10. EXISTS, NOT EXISTS, and correlation . . . 63
11. NOT IN with NULL: explain the counterexample . . . 63
12. UNION, INTERSECT, EXCEPT, and identity . . . 64
13. Every target: no required match is missing . . . 64
14. Remaining slide material and scope . . . 65
15. Query-writing checks and revision checklist . . . 66
Sources and preparation record . . . 67

01 Database Fundamentals: English Study Notes

CSC5041 · In-class Test 1 revision · 19 October 2026

Scope and learning goals

This chapter covers database fundamentals. ER diagram construction, relational algebra, and SQL query techniques will be covered separately. The main source is slides 17–57 of the lecturer's introductory deck; slide numbers refer to their order in the PPTX. Sections 7 and 8 supplement topics explicitly named in Week 1 of the teaching plan but not fully developed in this deck. They are not quotations from the lecturer's slides. Examples illustrate concepts; they are neither actual examination questions nor measured results.

After studying this chapter, you should be able to distinguish data, a database, a DBMS, and a database system; explain limitations of file-based processing; distinguish a schema from an instance; explain design stages, schema levels, and data independence; and identify the functions of the main DBMS components.

1. Four foundational concepts

Source: slides 17–18, 33, 41, and 45.

Term	Meaning	Course-enrolment example
Data	Recorded facts with implicit meaning	Student ID S01, the name Amy, course CSC5041
Database	A collection of related data modelling part of the real world	Student, course, and enrolment records
Database management system (DBMS)	Software for creating, maintaining, querying, and managing databases	PostgreSQL database management software
Database system	In the slides' simplified definition, the database together with its DBMS	Enrolment data and the DBMS managing it

The database contains the data; the DBMS manages the data. SQL is a language for interacting with relational databases, rather than a database or a DBMS. An application uses the DBMS to access data: a course-registration website is an application, not the DBMS itself.

Related data describes a meaningful application domain. It does not require every record to be directly linked to every other record.

Structured and unstructured data

Source: slide 34.

- **Structured data** follows a predefined organization, such as a student record with ID, name, and major.
- **Unstructured data** is not organized into a fixed record schema, such as free text, recordings, or images. Analysis commonly requires additional preparation.

Unstructured does not mean meaningless or completely without internal structure. Storing something in a computer file does not make it a relational table.

2. Why not manage everything with files?

Source: slides 35–40.

The lecture uses three files for course enrolment:

```
STUDENT(name, id, major)
COURSE(code, name, department)
SELECTION(student_id, course_code, term)
```

To check whether Amy takes a course, an application first finds her ID in the student file, then matches that ID, the course code, and the term in the selection file. Files can store the information. The difficulty is repeatedly implementing access, integrity, concurrency, and recovery logic in individual applications.

Limitation	Meaning	Illustrative example
Data redundancy	The same fact is stored repeatedly	Several departments store separate copies of a student's address
Data inconsistency	Copies of the same fact disagree	An address is updated in only one file
Difficult data access	New requests require new processing logic	Each reporting requirement needs another file-processing program
Data isolation	Data is scattered across files and formats	Student and course information use incompatible formats
Integrity problems	Rules are buried in separate application programs	One program checks nonnegative balances; another omits the check
Difficult atomic updates	A multi-step operation may finish only partly	Money is deducted before a crash prevents the corresponding credit
Concurrent-access problems	Overlapping operations interfere	Two withdrawals read the same old balance and overwrite each other's result

Redundancy can cause inconsistency, but duplicate copies need not disagree at every moment. The slides emphasize avoiding unnecessary redundancy. Deliberate, controlled replication is different from unmanaged duplication.

Here, **data isolation means separation across files and formats**. Do not confuse this use of the term with transaction isolation in later lectures.

Two examples you should be able to explain

A transfer and atomicity: transferring 100 from A to B requires both a debit and a credit. Either both changes take effect, or a failed operation undoes any partial changes. A debit without the credit is not an acceptable completed transfer.

Concurrent withdrawals and a lost update (supplementary terminology): start with a balance of 100. Two operations both read 100, each withdraws 50, and both write 50. If both withdrawals are accepted, the correct final balance is 0, but the stored result is 50. Interleaving operations and overwriting an old value cause the problem; a separate balance check alone is insufficient.

For this chapter, understand the problems. Detailed concurrency-control and recovery algorithms belong to later lectures.

3. What does a DBMS provide?

Source: slides 41–42, 44, and 47–48.

A DBMS supports defining structures, constructing and loading a database, querying data, inserting/deleting/updating data, and administering access.

Capability	Purpose
Data independence	Reduce application dependence on lower-level representations and storage
Efficient data access	Provide shared query and update mechanisms rather than requiring custom file scans in every application
Integrity	Maintain valid data states using declared constraints
Security	Control who can read or change particular data
Administration	Manage structures, permissions, and operation centrally
Concurrency and recovery	Coordinate shared access and recover to a consistent state after failures
Reduced development effort	Let applications reuse common data-management services

Integrity and security are different. Preventing an invalid enrolment record concerns integrity. Preventing a student from changing another student's grades concerns security. The DBMS supplies mechanisms, but the relevant rules must still be designed and configured correctly.

The slides also list timely response, modifiability, robustness, multiple-user support, and security as expectations. These are system goals, not automatic guarantees of performance or correctness merely from adopting a DBMS.

4. Data models, metadata, schemas, and instances

Source: slides 50 and 53; metadata explanation also draws on the diagram on slide 56.

Data model

A data model provides concepts and rules for describing database structure. It tells us what kinds of structures can represent data, rather than which records exist at a particular moment.

Level	Focus	Example
High-level or conceptual model	Application entities, attributes, and relationships	ER model
Implementation or record-based model	Structures suitable for logical database implementation	Relational, hierarchical, and network models
Low-level or physical model	Record formats, storage organization, and access paths	How records are arranged and located

The relational model is a type of model. A particular definition of the columns in Student is a schema using that model. The students currently stored are part of an instance.

Schema versus instance / state

```
Schema: Student(sid, name, major)

Instance at t1:
(S01, Amy, CS)
(S02, Ben, Math)

Instance at t2:
(S01, Amy, CS)
(S02, Ben, Math)
(S03, Chen, CS)
```

- Inserting the third student changes the **instance**, not the schema.
- Changing Amy's major to Math changes the **instance**, not the schema.
- Adding an email attribute changes the **schema**.
- A defined table with no records still has a schema and an empty instance.

Metadata is data describing data, such as table names, column names, types, and constraints. The system catalog or data dictionary stores such descriptions for the DBMS. Amy's name is application data, not metadata.

5. Three stages of database design

Source: slide 54.

Stage	Main question	Enrolment example
Conceptual design	What objects and business rules must the application represent?	Students, courses, and enrolment relationships
Logical design	How are they represented using the target data model?	Student, Course, and Selection relations and constraints
Physical design	How will data actually be stored and accessed efficiently?	File organization, indexes, and storage arrangements

Do not begin with disk formats before understanding the requirements. Conceptual design does not mean entering individual records.

These design stages and the three-schema architecture in Section 7 are related, but they are different classifications. Design stages describe a development process; schema levels describe abstraction and interfaces. Do not list the design stages as the names of the three schemas.

6. Database languages and users

Source: slides 49 and 55.

DDL and DML

Category	Purpose	Recognition examples; syntax comes later
Data Definition Language (DDL)	Define or modify structures and constraints	CREATE TABLE, ALTER TABLE, DROP TABLE
Data Manipulation Language (DML)	Retrieve, insert, delete, and modify data	SELECT, INSERT, DELETE, UPDATE

The introductory slides use DML broadly to include retrieval. Some other materials call SELECT a separate Data Query Language (DQL); that does not invalidate the classification used here.

Declarative / non-procedural: specify the desired result and let the DBMS determine how to obtain it. For example, request the names of CS students.

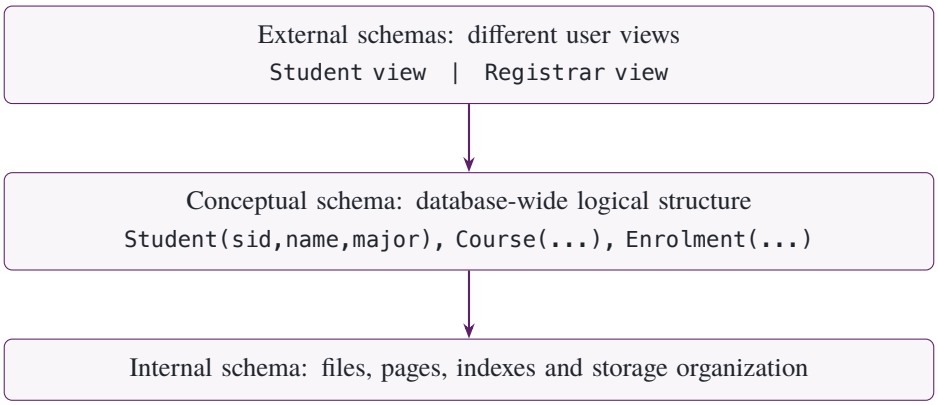
Procedural: specify a sequence of operations, such as reading each student record, checking its major, and outputting the name. SQL queries are generally declarative; later PL/pgSQL material adds variables, branches, and loops.

Who uses a database?

Role	Responsibilities
Database administrator (DBA)	Schemas, security and authorization, availability, recovery, and performance tuning
Application programmer	Programs that access the database to meet application requirements
End user	Queries or applications used to complete business tasks

These are distinct responsibilities, although one person may perform several roles in an organization.

7. Three-schema architecture [supplement]



Logical independence: change conceptual schema; preserve external schemas/applications.
Physical independence: change internal schema; preserve conceptual schema.

Scope basis: Week 1 on page 1 of the teaching plan. Definitions below supplement the slides rather than reproduce their wording.

Schema	What it describes
External schema	The view presented to a particular user group or application; several may exist
Conceptual schema	The database's overall logical structure, including data and constraints
Internal schema	Physical storage organization and access methods

Mappings connect the levels. An external schema can hide information or present computed information. Different views do not require separate database copies.

Definitions: UCT database course, Chapter 1, "ANSI/SPARC three-level architecture".

Original illustration: a registrar sees full enrolment details; a student sees only their own enrolment list. These are external views. Student, Course, Selection, and their constraints form the overall logical structure. File organization and access paths belong to the internal level.

This architecture is different from the presentation/business/database tiers of application deployment.

8. Data independence [supplement]

Scope basis: page 1 of the teaching plan and slide 42.

Type	What may change?	What should remain unaffected?
Physical data independence	Internal schema	Conceptual schema and higher-level application interfaces
Logical data independence	Conceptual schema	External schemas and the applications using them

Mappings may need adjustment. Data independence does not promise automatic compatibility with every possible structural change.

Definitions: Emory course material on the three schemas. For physical/logical separation, also see Oracle Database Concepts.

Classifying a change

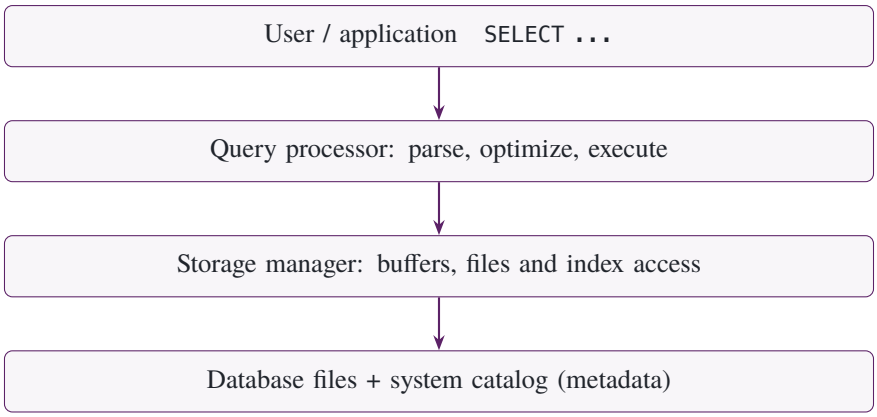
Example A (original): reorganize the underlying files while preserving Student's logical definition and the application's queries. This demonstrates physical data independence.

Example B (original): split the student structure into basic information and contact information, then adjust a view mapping so an existing application retains its former student-information interface. This demonstrates logical data independence, provided the information required by the old interface remains derivable. Arbitrary table splitting is not automatically harmless.

Example C (original): changing Amy's phone number changes a data value. It is not an example of logical data independence.

Use the order **user views** → **overall logical structure** → **physical storage**. Identify the level that changes, then explain which higher-level interface is protected.

9. Understanding the DBMS component diagram



Simplified conceptual flow; authorization, integrity, concurrency and recovery support request handling.

Source: slide 56; the original diagram was visually inspected. The explanation below is functional, not a guaranteed execution sequence for every product.

Component	Basic function
DDL compiler	Process structural definitions and provide their descriptions to the catalog
System catalog / data dictionary	Store metadata about database structures
Query compiler	Process a query into a representation suitable for subsequent execution
Query optimizer	Choose a suitable query execution plan
Runtime database processor	Handle database requests during execution

Component	Basic function
Stored data manager	Manage low-level data access and I/O
Concurrency-control / backup / recovery subsystems	Coordinate concurrent operations and support backup and recovery

For an interactive query, use this simplified flow:

```
User query → query compilation → optimization → runtime processing → data access →
↪  result
                                                    ↕
                                        catalog supplies metadata
```

The original figure also includes application precompilation, DML compilation, and host-language compilation. Recognize these as additional entry paths; compiler implementation details are outside this chapter.

Keep the viewpoints separate: schema architecture explains levels of data description; the component diagram explains which software modules handle requests.

10. Database types: recognize categories, not live rankings

Source: slides 20 and 51–52.

Relational databases mainly organize data as relations/tables. The slides also introduce key-value, document, graph, and column-family models. Examples in the deck include PostgreSQL and MySQL for relational systems, Redis for key-value storage, MongoDB for documents, and Cassandra for column-family storage. These examples identify models; they are not current market rankings or exhaustive descriptions of product capabilities.

NoSQL does not mean "no structure." A relational database is more than a collection of unrelated spreadsheets. The rest of this revision series primarily concerns relational databases.

11. Common distinctions

Pair	Distinction
Database / DBMS	Related data / software managing it
Schema / instance	Structural description / actual contents at a particular time
Data model / schema	Concepts for describing structures / a particular structure defined using them
Metadata / data	Descriptions of structures and related information / application facts
Redundancy / inconsistency	Repeated storage / conflicting copies
Integrity / security	Whether data obeys rules / who may access or modify it
DDL / DML	Define structures / retrieve and change contents
Design stages / schema levels	Development process / levels of data abstraction

Pair	Distinction
Physical / logical independence	Shield higher levels from storage changes / shield external interfaces from overall logical changes
Atomicity / concurrency	Whether one unit of work takes effect completely / how overlapping operations share data

12. Revision checklist and short-answer language

These are learning goals, not a formal practice test or an official mark allocation.

- ☐ Explain the four foundational concepts with an example of each.
- ☐ Explain at least four file-based limitations using cause and consequence.
- ☐ Distinguish the problems illustrated by a failed transfer and overlapping withdrawals.
- ☐ Classify record insertion, value updates, and adding a column as instance or schema changes.
- ☐ Distinguish data models, design stages, and schema levels.
- ☐ Identify the level affected by a change and explain the relevant form of independence.
- ☐ Explain DDL, DML, the DBA's role, and the functions of core DBMS components.

Useful short-answer formulations:

> A database is a collection of related data, whereas a DBMS is the software used to create and manage it.
>
> A schema describes the database structure; an instance contains the actual data at a particular time.
>
> Physical data independence allows storage changes without changing the conceptual schema.
>
> Logical data independence allows conceptual-schema changes while preserving existing external interfaces through suitable mappings.
>
> Atomicity means that a transaction's changes take effect as a whole or do not take effect at all.

Sources and preparation record

- Lecturer's introductory deck: Week 1A - IntroductionNEW.pptx, mainly slides 17–57; component diagram on slide 56.
- Lecturer's plan: CSC5041_Teaching_Plan.pdf, page 1, Week 1 and Test 1.
- Supplementary web references support the architecture and independence definitions in Sections 7–8. Original examples are labelled. External references do not determine the lecturer's intended examination depth.
- LaTeX edition prepared on 20 October 2026; original evidence status is retained. No new database execution is claimed.
- Input versions: introductory deck SHA-256 `d36c2d15a77083931046168738d794558c2e9821d4527 01f64ef524e45e87a7c`; teaching plan SHA-256 `b62c7356961bf10585f2e7d5fbf29bcb8401970a9 9d09d80fe6675fa63cea463`.

02 ER Modelling: English Study Notes

CSC5041 · In-class Test 1 revision · 19 October 2026

Scope and learning goals

This chapter follows 01 Database Fundamentals. It covers Week 2 on page 1 of the teaching plan and Question 1(a)–(b) on page 1 of Tutorial 1. The main source is the lecturer's Week 1B - Conceptual DB Design.pptx. All slide numbers below refer to their order in that file.

After studying this chapter, you should be able to identify entities, attributes, keys, and relationships from requirements; recognize weak entities; distinguish cardinality from participation; and draw ER diagrams using the lecture notation. Relational integrity constraints belong to Chapter 03; systematic ER-to-relational mapping belongs to Chapter 04. Explanatory supplements and original examples below do not establish additional examination requirements.

Slide 70 explicitly states: **"You must use the notation in the lecture for this course."** This chapter uses the lecture's Chen-style notation rather than substituting crow's foot notation.

1. What problem does ER modelling solve?

Source: slides 2–10, 54–56, and 69–70.

The Entity-Relationship model organizes application requirements into **entities, attributes, relationships, and constraints**. It belongs to conceptual design.

```
Application requirements → identify objects and connections → ER diagram and
↪  constraints
                                                                    ↓ Chapter 04
                                                          relations and primary/foreign
                                                          ↪  keys
```

For example, "Every book must be published by exactly one publisher" supplies two entity types, Book and Publisher; a relationship, PUBLISH; and a constraint requiring exactly one publisher for each book. Drawing two boxes and an unlabelled line omits the business rule.

Conceptual design first asks what information the application must express. This chapter does not begin with SQL table definitions or indexes, or mechanically add foreign keys as ER attributes.

2. Entities, entity types, and entity sets

Source: slides 10–14 and 24–27.

Term	Meaning	Example
Entity	A distinguishable physical or conceptual object	Amy, the student with ID S01
Entity type	A definition of the common attributes and structure of a class of entities	Student with attributes sid, name, etc.

Term	Meaning	Example
Entity set	The collection of entities of a type at a particular time	All students currently registered
Attribute	A property of an entity or relationship	A student's name; a grade for an enrolment
Domain / value set	The set of permitted values for an attribute	The allowed date values for an attribute

An entity type describes structure; an entity set describes current contents. Connect this distinction to schema versus instance in Chapter 01. Entities need not be physical objects: courses, accounts, and appointments can also be entities.

Entity or attribute? Ask whether the application must identify the object independently, record its own properties, and relate it to other objects. A publisher's name alone might be an attribute. If the application needs a publisher ID, publisher name, and the books it publishes, Publisher is better represented as an entity type. Being a noun alone does not make something an entity type.

3. Attribute types: distinguish three dimensions

Source: slides 14–23 and 27–28.

Dimension	Type	Meaning and example
Whether meaningful components are needed	Simple / atomic	Not subdivided in the current design, such as salary
Whether meaningful components are needed	Composite	Contains components with independent meanings, such as address(city, street, postcode)
Number of values per entity at one time	Single-valued	One dateOfBirth per student
Number of values per entity at one time	Multivalued	Several phoneNumber values per student
How the value is obtained	Stored	dateOfBirth is recorded directly
How the value is obtained	Derived	age is calculated from dateOfBirth and a reference date

These are not six mutually exclusive categories. An address can be **single-valued and composite**. If several addresses are allowed per person, it can be **multivalued and composite**. Slide 15 contrasts multivalued attributes with simple attributes; for clarity, assess decomposition and number of values separately. Simple and single-valued are not synonyms.

Composite versus multivalued

```
Composite:   one address = (city, street, postcode)
Multivalued: phone numbers = {130..., 131...}
```

Branches from a composite attribute describe **components**. A double oval means that an entity may have **more than one value** for the attribute. Slide 21 explains that subdivision is unnecessary if an address is used only as a whole; the choice depends on the operations required.

Stored versus derived

Age depends on date of birth and a reference date. Manually updating age can make it inconsistent with the birth date, so the lecture's conceptual treatment uses age as a derived attribute and dateOfBirth as a stored attribute. This describes the conceptual meaning, not a required caching implementation for a real product.

Missing values: NULL

Slide 28 uses NULL for an unknown or inapplicable attribute value: a person's height may be unknown, or a person may have no college degree. NULL is not the number zero or a known empty string. SQL comparisons involving NULL belong to Chapter 06.

4. Keys: check uniqueness and minimality separately

Source: slides 29–32.

Type	Definition
Superkey	A set of attributes that uniquely identifies an entity; redundant attributes are allowed
Candidate key	A minimal superkey: removing any attribute destroys the guarantee of uniqueness
Primary key	A candidate key selected by the designer
Composite key	A key consisting of more than one attribute

Minimal does not mean minimum size. A type can have both a one-attribute candidate key and a two-attribute candidate key, provided neither contains a redundant attribute.

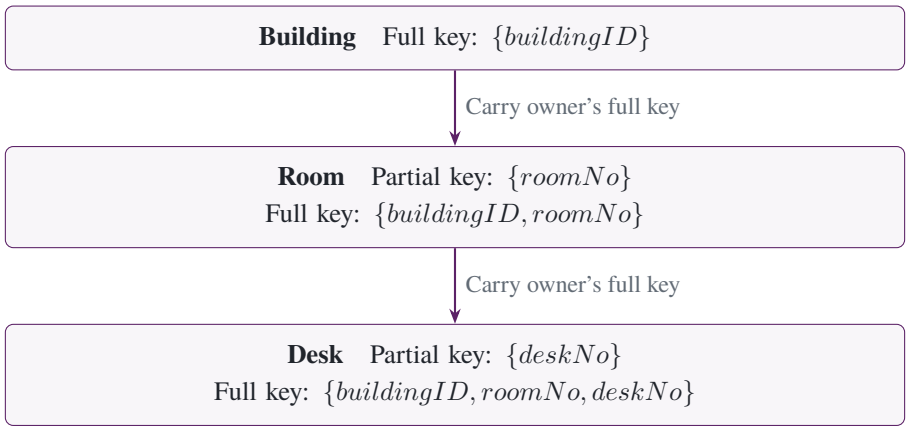
Original example: suppose both sid and email are guaranteed unique in Student.

- `{sid}` and `{email}` are candidate keys; `{sid}` may be selected as the primary key.
- `{sid, name}` is a superkey but not a candidate key, because name is redundant.
- Three current students having different names does not establish `{name}` as a key. A key must be unique in every state permitted by the requirements.

Components of a composite key need not be individually unique. If a local number is unique only within its region, `{regionCode, localNumber}` may jointly identify an object. A key containing several attributes is not a multivalued attribute.

An artificial identifier can be introduced when natural attributes cannot reliably distinguish objects. Adding an ID does not by itself express all other business constraints.

5. Strong entities, weak entities, and partial keys



Original example. Room numbers are local to a building; desk numbers are local to a room. Desk references Room with $(buildingID, roomNo)$, not $roomNo$ alone.

Source: slides 33–35; local numbering also appears in Tutorial 1, Question 1(a).

Term	Identification rule
Strong entity	Has its own key and can be identified independently
Weak entity	Lacks a complete key of its own; identification requires the owner's identifier and its own partial key
Owner entity	Supplies the context required to identify a weak entity
Partial key / discriminator	Distinguishes weak entities belonging to the same owner, without guaranteeing uniqueness across owners
Identifying relationship	Connects a weak entity to its owner

The phrase "partially unique" on slide 34 should be read as **unique within an owner's scope**. It does not mean that the key identifies only some of the entity's attributes.

A chain with two weak entity types

The following explains the numbering rules in Tutorial 1; it is not a complete solution to the tutorial.

```
Owner: ownerID is globally unique
  └─ Pet: petNo is unique only within that Owner
       └─ Treatment: treatmentNo is unique only within that Pet

Full identifier of Pet       = (ownerID, petNo)
Full identifier of Treatment = (ownerID, petNo, treatmentNo)
```

Pet has partial key petNo; Treatment has partial key treatmentNo. Treatment's immediate owner, Pet, is itself weak, so identification must follow the owner chain to obtain the full context.

Here each Pet belongs to exactly one Owner, and each Treatment belongs to exactly one Pet. A weak entity has **total participation** in its identifying relationship. This does not imply that every owner must

have a weak entity. Tutorial 1 explicitly requires each Owner to have at least one Pet, but does not require every Pet to have a Treatment.

Clarifying the slides: "a weak entity does not have its own primary key" means that it lacks an **independent complete key of its own**. A relation produced later can still have a primary key combining the owner key and partial key; Chapter 04 covers that mapping.

Dependency alone does not establish weakness. An employee may be required to work in a department, but can still be strong if empID identifies the employee independently. The issue is dependence on the owner for identification.

6. Relationships, relationship attributes, and roles

Source: slides 38–42 and 48–53.

A relationship type describes an association between entities. A relationship instance contains particular participants. ENROLLED_IN(Student, Course) is a relationship type; "S01 takes C01" is an instance.

Where should an attribute go?

In the WORKS_ON example on slide 48, fraction is the proportion of a researcher's time devoted to a particular project:

```
(Researcher R1, Project P1) → fraction = 0.4
(Researcher R1, Project P2) → fraction = 0.6
```

The value describes a **particular researcher–project combination**, so it belongs to WORKS_ON. Attaching it only to Researcher would not distinguish projects; attaching it only to Project would not distinguish researchers. Similarly, borrowing dates describe a borrowing association.

Several relationships can connect the same types

In Tutorial 1, Clinic and Veterinarian participate in both WORKS_AT and MANAGES. Employment and management have different meanings and should not become a single unlabelled line.

"The manager must work at that clinic" further requires **every MANAGES pair to also be a WORKS_AT pair**. Drawing both relationships and assigning their individual cardinalities does not fully express this rule; add a written constraint.

Roles and relationship degree

A binary relationship has two participating roles; a ternary relationship has three. Slide 50 illustrates Researcher, Student, and Course. Slide 53 shows one Person type participating in separate Student and Researcher roles.

Explanatory supplement: the number of roles need not equal the number of distinct entity types. EMPLOYEE can participate in SUPERVISES as both supervisor and subordinate, giving a binary recursive relationship. Label roles when a type participates more than once.

Slide 52 advises beginners to try binary representations. This is a modelling heuristic, **not a theorem that every ternary relationship can be decomposed without loss into three binary relationships**. Check whether the original association among all three participants can still be recovered. This chapter mainly focuses on binary relationships.

7. Cardinality constraints: ask about both maxima

Source: slides 43–45.

For a binary relationship A—R—B, fix one A and then one B, and ask how many entities on the other side each can be associated with at most.

Cardinality ratio A:B	Maximum B entities for one A	Maximum A entities for one B
1:1	One	One
1:N	Many	One
M:N	Many	Many

Original example: a Department can have many Employees, and each Employee belongs to at most one Department. Therefore Department:Employee is 1:N.

The lecture's Chen notation is `Department` — `1` — `WORKS_FOR` — `N` — `Employee`. **The 1 beside Department does not mean one employee per department.** It means at most one Department for a given Employee. Write the two verbal statements before positioning the labels.

The example on slides 44–45 works the same way: a Grant supports only one Project, while a Project can receive support from many Grants. Thus Project:Grant = 1:N, or Grant:Project = N:1 when the order is reversed.

`N` and `M` permit many; they do not specify an exact current count or require at least two. **A 1:N ratio alone does not say that every employee must have a department.**

8. Participation constraints: ask about both minima

Source: slides 46–47; notation on slide 57.

Constraint	Requirement for each entity of the type	Lecture notation
Total participation	Must participate in at least one relationship instance	Double line between entity and relationship
Partial participation	May participate in none; the minimum is zero	Single line

Partial participation does not require some entities to remain uninvolved; it **allows** that possibility. Every current entity participating is not evidence that the rules require total participation.

Combining minima and maxima helps check the design. The following `(min,max)` notation is a reading aid for written tables. Formal diagrams still use the lecture's cardinality labels and single/double lines.

Wording	Participation count for the entity under discussion
Zero or one / at most one	(0,1)
Exactly one	(1,1)
Zero or more	(0,N)
One or more / at least one	(1,N)

Suppose every Employee must work in exactly one Department, but a Department may temporarily have

no Employees. Employee has `(1,1)` and Department has `(0,N)`. Department:Employee is 1:N; Employee participates totally and Department participates partially.

Watch the direction: an entity's `(min,max)` above counts its possible counterparts. This is a different reading convention from the placement of Chen cardinality labels. Do not copy positions mechanically.

9. Lecture notation reference

Source: slides 35–37, 45, 47, and 57; the original diagrams were visually inspected.

Meaning	Symbol
Regular / strong entity type	Single rectangle
Weak entity type	Double rectangle
Regular relationship type	Single diamond
Identifying relationship	Double diamond
Regular attribute	Single oval
Key attribute	Solid underline under the name; indicate all components of a composite key
Partial key	Dashed underline under the name, illustrated on slide 35
Multivalued attribute	Double oval
Composite attribute	An oval connected to ovals for its components
Derived attribute	Dashed oval
Total / partial participation	Double / single line
Cardinality ratio	Labels 1, N, or M at the relationship ends

Double rectangles, diamonds, ovals, and connecting lines have different meanings. **A double line means total participation, not many-to-many. A double oval means multivalued, not composite.** An underline for a key is also different from the dashed boundary of a derived attribute.

10. Worked example: library requirements to an ER diagram

Source: slides 58–68. The following restates the lecture requirements and provides a study redraw; it is not a new practice examination question.

Step 1: entities and attributes

Entity type	Key	Other attributes
Book	bookID	title, price, availability
Reader	readerID	name, phoneNumber, address(street, suburb)
Publisher	publisherID	name
Author	authorID	name, phoneNumber, dateOfBirth

The full requirements on slides 58–60 do not give Reader a birth date, although the step-by-step example on slide 62 introduces dob. This chapter follows the full requirements and final diagram on slide 68, omitting Reader.dob. Author's date of birth is explicitly required. This is a local inconsistency in the slides, not a reason to silently add a requirement.

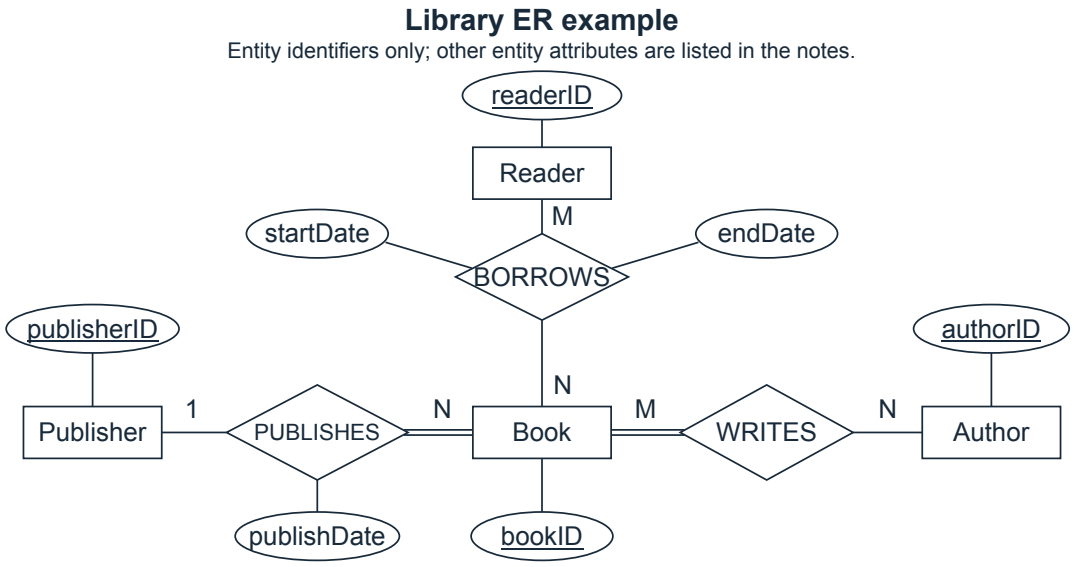


Figure 1. Library ER study diagram: identifiers, relationship attributes, cardinalities, and participation

Step 2: state each relationship in both directions

Relationship	Counterparts for each left entity	Counterparts for each right entity	Cardinality and participation	Relationship attributes
Reader—BORROWS—Book	A Reader may borrow zero or more Books	A Book may be borrowed by zero or more Readers	M:N; both partial	startDate, endDate
Publisher—PUBLISHES—Book	A Publisher may publish zero or more Books	A Book must have exactly one Publisher	1:N; Publisher partial, Book total	publishDate
Author—WRITES—Book	An Author may write zero or more Books	A Book must have one or more Authors	M:N; Author partial, Book total	None

The Book–Reader association follows the scope of the lecture requirements. It does not establish that one physical copy can be lent to several readers simultaneously.

Step 3: draw and read back the model

This redraw shows **only key attributes of entities**. Other entity attributes appear in the table above. The date attributes belong to relationship diamonds. Check the double lines from Book to PUBLISHES and WRITES, then the single lines at both ends of BORROWS, to distinguish the three participation rules.

Model boundary (explanatory supplement): a regular binary relationship instance is determined by its participating entity pair. If the application must preserve separate records of the same Reader repeatedly borrowing the same Book, dates attached to a Reader–Book relationship alone do not distinguish those repeated events. Introduce a Loan entity and specify how it is identified. The lecture example does not fully develop this history requirement; this supplement is not an additional compulsory answer imposed

on the original exercise.

11. A procedure for modelling written requirements

Source: slides 54–55 and 58–70; the steps below organize the lecture material into a working procedure.

1. **Read all requirements, then identify entities.** Define each candidate type and decide which objects require independent identification.
2. **List and classify attributes.** Identify composite, multivalued, derived, and potentially missing information; find the entity keys.
3. **Check weak entities.** Is a number unique only within an owner? Record the owner, partial key, and full identification chain.
4. **Name each relationship.** Several relationships may connect the same types; label roles when a type participates more than once.
5. **Write minima and maxima in both directions.** Derive cardinality and participation from them; a bare "1:N" is insufficient.
6. **Place relationship attributes.** Decide whether each property describes one object or an association among participants.
7. **State rules the diagram cannot fully express.** Examples include MANAGES pairs being WORKS_AT pairs, time ordering, and numbering sequences. Label assumptions where requirements are silent.
8. **Read the whole diagram back into sentences.** Check every box, connection, and label against the requirements for both missing and unjustified constraints.

Slide 55 gives four design criteria: correctness, completeness, consistency, and readability. Several designs may be reasonable, but "no single best design" does not excuse missing constraints.

12. Common distinctions

Pair	Distinction
Entity / entity type / entity set	One object / common structural definition / current collection
Simple / single-valued	Whether meaningful subdivision is needed / whether one value is allowed at a time
Composite attribute / composite key	An attribute with components / several attributes jointly identifying an entity
Composite / multivalued	Components of one value / several values for one entity
Superkey / candidate key	Unique / unique with no redundant attribute
Partial key / full identifier	Unique within one owner / complete identification with owner context
Weak entity / existence dependency	Weakness requires identification through an owner; existence dependency alone is insufficient
Cardinality / participation	Maximum related entities / whether every entity must participate
Double line / double oval	Total participation / multivalued attribute
Entity attribute / relationship attribute	Property of an object / property of an association
ER relationship / relational relation	Association in a conceptual model / a relation (table) in the relational model

13. Revision checklist and short-answer language

These are learning goals, not a formal practice test or an official mark allocation.

- ☐ Distinguish an entity, entity type, and entity set using examples.
- ☐ Classify attributes separately by decomposition, number of values, and derivation.
- ☐ Explain why a minimal candidate key need not have the smallest number of attributes.
- ☐ Give a weak entity's owner, partial key, identifying relationship, and full identifier.
- ☐ State minimum and maximum participation counts in both directions for every relationship.
- ☐ Use key underlines, double rectangles, double diamonds, double ovals, and single/double lines correctly.
- ☐ Explain why time fractions and borrowing dates can be relationship attributes.
- ☐ Translate the library diagram back into requirements and identify its boundary for repeated borrowing history.
- ☐ State assumptions explicitly when a constraint cannot be determined from the requirements.

Useful short-answer formulations:

> An entity type describes a collection of entities that share the same attributes.
>
> A candidate key is a minimal superkey: no proper subset is still a superkey.
>
> A weak entity is identified by its owner's identifier together with its partial key.
>
> Cardinality constrains the maximum number of related entities; participation specifies whether every entity must participate.
>
> Total participation requires every entity of that type to participate in at least one instance of the relationship.
>
> The attribute belongs to the relationship because its value describes a particular combination of participating entities.

Sources and preparation record

- Lecturer's deck: Week 1B - Conceptual DB Design.pptx, 70 slides. Text was read; diagrams on slides 35–36, 45, 47, 51, 53, 57, and 68 were visually inspected. The library example follows slides 58–68; the SVG is a study redraw omitting non-key entity attributes.
- Scope: CSC5041_Teaching_Plan.pdf, page 1, Week 2, and page 2, Tutorial 1. Supporting exercise: Tutorial1.pdf, page 1, Question 1(a)–(b). This chapter supplies neither a complete tutorial solution nor a practice paper.
- Clarifications concerning attribute classification, partial keys, roles, ternary decomposition, and repeated borrowing are identified in the relevant sections. They are not presented as verbatim lecture statements.
- LaTeX edition prepared on 20 October 2026; original evidence status is retained. No new database execution is claimed.
- Input SHA-256: ER deck `1f2bc2b2242208b0d3f0c650341521c65cba833cffe5da82fc27d0ddef7fc734`; Tutorial 1 `316e8b6bb9581a5ecae3fb2f449e395a4ea8a19b4ee16c4191fc20a6079d2c44`; syllabus teaching plan `31380a7fe6504e46541b6db0bb2f24859c69188bfeb9681dca73354ebeb68295`. These are the versions read this turn, rather than hashes copied from Chapter 01.

03 Relational Model and Integrity Constraints: English Study Notes

CSC5041 · In-class Test 1 revision · 19 October 2026

Scope and learning goals

This chapter follows 02 ER Modelling. It covers "The Relational Data Model" and "integrity constraints" in Week 3 on page 1 of the teaching plan. The main source is the lecturer's Lecture2A.pptx, slides 4–32. Unless another file is named, slide numbers below refer to their order in that deck.

After studying this chapter, you should be able to interpret relation schemas and instances, explain set properties, identify keys and foreign-key directions, distinguish key constraints from entity and referential integrity, and assess insertions, deletions, and modifications. ER mapping begins on slide 33 and belongs to Chapter 04; relational algebra belongs to Chapter 05; SQL implementation details belong to Chapter 06.

The student-enrolment data below is an **original teaching example**, not real student information, measured output, or examination questions. Clarifications of simplified slide statements are labelled explicitly.

1. From the ER model to the relational model

Source: slides 4–7 and 32.

Chapter 02 expresses application meaning through entities, attributes, and relationships. The relational model organizes information into a collection of interconnected **relations**. A relation can be visualized as a table; it is **not the same thing as an ER relationship** connecting entities.

Mathematical term	Common data terminology	Meaning
Relation	Table	A set of tuples sharing an attribute structure
Tuple	Row / record	Values for a collection of attributes
Attribute	Column / field	A named property
Domain	Allowed values	The set of values permitted for an attribute

For example, `Student(sid, name, email)` is a relation schema. `(S01, Amy, amy@example.test)` is a tuple written in that attribute order. The enrolment association between Student and Course can itself be represented by an Enrolment relation; placing tables next to each other does not encode the association.

2. Schemas, instances, degree, and cardinality

Source: slides 8 and 32. The formal notation and degree/cardinality terminology below are explanatory supplements.

One relation

```
Relation schema: Student(sid, name, email)
```

```
A relation instance:
sid   name   email
S01   Amy    amy@example.test
S02   Ben    ben@example.test
```

A relation schema specifies the relation name, attributes, and their domains. An instance is the actual set of tuples at a particular time. Common notation is `R(A1, ..., An)` and `r(R)`, or simply `r` when the schema is clear.

In the classical definition without NULL, each tuple takes one value from each attribute domain, so $r \subseteq \mathsf{dom}(\mathsf{A1}) \times \ldots \times \mathsf{dom}(\mathsf{An})$. This specifies where values come from and which combinations are included. It does not require storing every possible combination.

Term	What it counts	Example above
Degree / arity	Attributes	3
Cardinality	Current tuples	2

An empty instance still has a schema: if Student has no rows, its degree remains 3 and its cardinality is 0. Here cardinality means **the actual number of rows**. The ER cardinality ratio in Chapter 02 constrains **the permitted number of associated entities**. Keep the meanings separate.

The database as a whole

Slide 32 defines a relational database schema as **a collection of relation schemas together with integrity constraints**. Listing the columns of Student, Course, and Enrolment alone does not fully describe the database schema; primary keys, foreign keys, and other rules also matter.

A database state contains instances of its relations. It is valid only when all relevant constraints hold. Invalid states discussed below explain why operations should be rejected; they are not states the system is expected to preserve.

3. Basic properties of a relation

Source: slides 8–10. The SQL comparison additionally uses SQLpart1, slides 31–32.

Property	Interpretation
Tuples are unordered	Swapping displayed rows does not produce a different relation
Complete tuples do not repeat	A classical relation is a set; identical tuples are not separate elements
Each attribute position contains one atomic value	The course's flat relational model does not directly retain composite or multivalued ER attributes
Attributes are distinguished by names	Attribute names must distinguish columns within one relation; different relations may reuse names
Rearranging displayed columns preserves meaning	Move each column's name and its values together; do not arbitrarily exchange values within a tuple

No semantic column order does not mean arbitrary value order

Slide 10 rearranges PLAYER's column headings together with their corresponding values, preserving the relation. With named attributes, what matters is which value belongs to which attribute.

However, under `Student(sid, name)`, `(S01, Amy)` differs from `(Amy, S01)`. Positional tuple notation must follow the declared attribute order. Rearranging the display is different from exchanging attribute values.

Atomic values and ER attributes

In Chapter 02, `address(city, street, postcode)` is composite and `{phoneNumber}` is multivalued. Slide 8 excludes these structures as direct attributes in the course's flat relations. Atomicity depends on the domains and operations adopted by the model; it does not mean that a string cannot be divided into characters. Chapter 04 covers the conversion.

Classical relations and SQL tables

Classical relations use set semantics. SQLpart1, slides 31–32, separately explains that SQL can retain duplicates and that DISTINCT removes duplicate query results. "Relations have no duplicate tuples" therefore does not mean that every SQL table automatically forbids duplicates. SQL syntax and explicit constraint declarations belong to Chapter 06.

4. Superkeys, candidate keys, and primary keys

Source: slides 12–15; continues Section 4 of Chapter 02.

Key	Condition	Original example: sid and email are each unique and mandatory
Superkey	Uniquely determines a tuple	`{sid}`, `{email}`, `{sid, name}`
Candidate key	A superkey containing no redundant attribute	`{sid}`, `{email}`
Primary key	The selected candidate key	Choose `{sid}`
Composite key	Consists of several attributes	Course's `{dept, courseNo}` in Section 8

Minimality asks **whether removing an attribute destroys guaranteed uniqueness**. It does not mean selecting the smallest key among all candidate keys. A relation can have several candidate keys; choosing a primary key does not remove the uniqueness requirements of the others.

Slide 13 explicitly **assumes that no two people share a name** before treating PLAYER.Name as a candidate key. Three different names in a sample do not establish that names are always unique in reality.

Why are key constraints needed if tuples cannot repeat?

```
(S01, Amy, amy@example.test)
(S01, Ada, ada@example.test)
```

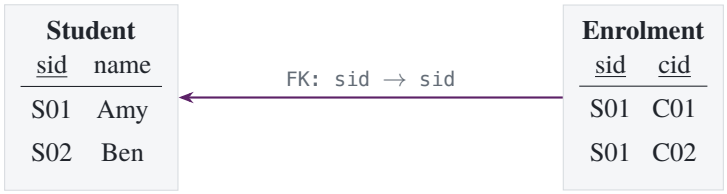
These complete tuples differ, so set membership alone does not exclude their coexistence. They share sid, however, and violate a key constraint if sid is the primary key. **No duplicate complete rows** and **uniqueness of a specified key** are different requirements.

Interpreting `2^n − 1` on slide 15

The slide lists all nonempty attribute subsets. An n-attribute set has `2^n − 1` such subsets, **but not every subset uniquely identifies tuples**. This is not an unconditional formula for the number of superkeys.

Original counterexample: R(A,B,C) has only candidate key `{A}`; B, C, and `{B,C}` do not guarantee uniqueness. Its superkeys are `{A}`, `{A,B}`, `{A,C}`, and `{A,B,C}`: four, not seven. All nonempty subsets are superkeys only under additional conditions, such as every individual attribute already being a key.

5. Foreign keys: direction, repetition, and composite references



Primary key: unique and non-NULL
$PK(Student) = \{sid\}$

Composite PK: pair unique; sid may repeat
$PK(Enrolment) = \{sid, cid\}$

$$\pi_{sid}(Enrolment) \subseteq \pi_{sid}(Student)$$

Here sid is part of the PK, so non-NULL; a general FK's nullability depends on constraints.

Original example; only the sid reference is shown. cid must also reference Course's key.

Source: slides 16–19.

A foreign key consists of one or more attributes in the referencing relation whose values reference a key in the referenced relation. This lecture section defines references to **primary keys**. This chapter follows that scope rather than claiming to give the complete rules of every SQL system.

```
Referencing relation                        Referenced relation
Enrolment.sid                  ————————→    Student.sid
Enrolment.(dept, courseNo)     ————————→    Course.(dept, courseNo)
```

Write each reference as **referencing attribute group → key attribute group in the target relation**. Attribute names need not match, but corresponding domains must be compatible. Matching names or simply using strings on both sides does not automatically establish a foreign key.

Foreign-key values need not be unique

Several enrolment rows can contain the same sid because one student may take several courses. Student.sid must be unique, but Enrolment.sid need not be unique merely because it is a foreign key. An attribute can also belong to both a foreign key and a primary key.

A composite foreign key must match as a combination

If Course contains only `(CS,101)` and `(Math,201)`, `(CS,201)` **does not** reference either tuple. CS and 201 occur separately, but not together in the corresponding key positions of **one target tuple**.

Likewise, `(Department, Degree)` on slide 19 is **one composite foreign key**, not two independent foreign keys referencing COURSE. Omitting either component may fail to identify the course.

6. NULL and the limits of foreign-key rules

Source: slides 18, 20, 22, and 30. Chapter 02 explains NULL as unknown or inapplicable information.

Location	Rule used in this chapter
Any component of a primary key	Cannot be NULL, even when other components have values
Non-primary-key attribute	Nullability depends on its mandatory-value constraints
Nullable foreign key	Can use NULL for no reference only when other constraints permit it
Foreign-key attribute also in a primary key	The foreign-key role does not override primary-key non-nullability

Slides 18 and 22 require a composite foreign key to be either entirely NULL or entirely non-NULL and matching a referenced key. This chapter follows that explicit lecture rule. It does not claim that this is the default behaviour of every SQL implementation.

Original small example: `Plan(planID, dept, courseNo)` has primary key planID and a nullable foreign key `(dept,courseNo)` referencing Course from Section 5, using the rule above.

Foreign-key value	Decision
`(CS,101)`	Valid: a complete match exists
`(NULL,NULL)`	Valid assuming both columns permit missing values and no other rule requires a reference
`(CS,NULL)`	Invalid under the lecture's all-NULL-or-all-non-NULL rule
`(CS,201)`	Invalid: this combination does not exist

Permission for a null reference does not establish that the business permits no association. If every record must reference a target, the foreign key must additionally be non-NULL. Merely having a nullable foreign key does not fully encode total participation from Chapter 02.

7. What do the three integrity constraints check?

Source: slides 20–22 and 32.

Constraint	Problem prevented	Typical violation
Key constraint	Distinct tuples share a candidate-key value	Two students have the same sid, or different sid values but the same unique email
Entity integrity	A primary key cannot completely identify a tuple	One component of a composite primary key is NULL
Referential integrity	A foreign key references a nonexistent target key	An enrolment references a student absent from the database

Primary-key **uniqueness** belongs to key constraints; primary-key **non-nullability** belongs to entity integrity. They jointly support identification, but distinguish them in answers. One operation may violate several rules; finding one violation does not establish that no others exist.

Domain and business constraints [explanatory supplement]

Domains on slides 6 and 8 restrict allowed values. For example, grade must be an allowed grade or permitted missing value; an integer identifier cannot arbitrarily contain free text. Other business rules may require nonnegative fees or a start date no later than the end date.

The three constraints emphasized on slide 20 are **not an exhaustive list of every possible database constraint**. Valid primary and foreign keys alone do not guarantee a correct business state.

8. Running example: schema, constraints, and initial state

Everything in this section is an original teaching example. The schemas are given inputs; this is not an ER-mapping demonstration.

Schema and constraints

Relation schema	Primary key (PK)	Other candidate keys	Foreign keys (FK)
Student(sid, name, email)	`{sid}`	`{email}`	None
Course(dept, courseNo, title)	`{dept, courseNo}`	No others specified here	None
Enrolment(sid, dept, courseNo, term, grade)	`{sid, dept, courseNo, term}`	No others specified here	`{sid}` → `Student.sid`; `{dept,courseNo}` → `Course.(dept, courseNo)`

The example requires all three Student attributes and all three Course attributes. sid, email, dept, title, name, and term are strings; courseNo is a positive integer. Allowed term values are `2026F` and `2027S`. grade permits A, B, C, D, F, or NULL, with NULL meaning that a grade is not yet available. Every primary-key component is non-NULL.

A student may take several courses and may repeat a course in another term. **At most one enrolment exists per student, course, and term.** This is the rule represented by Enrolment's four-attribute primary key. Neither dept nor courseNo alone identifies Course tuples.

Initial state

Student:

sid	name	email
S01	Amy	amy@example.test
S02	Ben	ben@example.test

Course:

dept	courseNo	title
CS	101	Database Basics
Math	201	Linear Algebra

Enrolment:

sid	dept	courseNo	term	grade
S01	CS	101	2026F	A
S02	CS	101	2026F	NULL

This state satisfies the stated constraints: both Student candidate keys are unique, primary keys are complete and non-NULL, and every foreign-key value has a target. Repetition of Enrolment.sid does not determine referential validity; the issue is whether it references an existing student.

Each operation below is assessed independently against this initial state, not executed in sequence. The decisions are static deductions from the constraints, not results of database execution.

9. Insertion: check the new tuple and reference targets

Source: slides 21–28. Operations below use the original data from Section 8.

Proposed operation	Decision and reason
Insert `(S01, Ada, ada@example.test)` into Student	Invalid: sid already exists; key constraint violation
Insert `(S03, Chen, amy@example.test)` into Student	Invalid: candidate key email repeats; checking only the primary key is insufficient
Insert `(S03, CS, 101, 2026F, A)` into Enrolment	Invalid: Student has no S03; referential integrity violation
Insert `(S01, CS, 201, 2026F, B)` into Enrolment	Invalid: Course has no `(CS,201)`; components cannot be matched separately
Insert `(S01, CS, 101, NULL, B)` into Enrolment	Invalid: primary-key component term is NULL; entity integrity violation
Insert `(S01, CS, 101, 2026F, B)` into Enrolment	Invalid: the four-attribute primary key already exists, even though grade differs
Insert `(S01, CS, 101, 2027S, NULL)` into Enrolment	Valid: a different term gives a different key, both foreign keys have targets, and grade permits NULL

A useful order is **domains and mandatory values** → **all candidate keys** → **primary-key non-nullability** → **each foreign-key group**. If asked for all violations, complete all relevant checks.

In the lecturer's examples, slide 24 inserts the existing Person#=2 into RESEARCHER to demonstrate a key conflict. Slide 26 inserts `(Comp.Sci., NULL)` into COURSE to demonstrate a missing composite-primary-key component. Slide 28 inserts an ENROLMENT with nonexistent Supervisee=6 to demonstrate a missing reference target.

10. Deletion: follow references in the reverse direction

Source: slides 29–30.

When deleting, check not only the tuple itself, but also **whether other tuples reference its key**.

Proposed deletion from the initial state	Effect of deleting without further action
Student S01	An Enrolment references S01; leaving that reference unchanged violates referential integrity
Course `(CS,101)`	Two Enrolment tuples reference it; leaving them unchanged violates referential integrity
Course `(Math,201)`	No tuple references it here; the remaining state satisfies the stated constraints
S01's Enrolment tuple	No other relation references Enrolment here; the remaining state satisfies the stated constraints

"Primary keys cannot be deleted" is therefore incorrect. Check dependencies and declared handling rules.

Slide 30 discusses these responses:

- **Reject deletion:** retain the referenced tuple and avoid dangling references.
- **Cascade to referencing tuples:** delete related tuples, checking whether further relations reference them.
- **Set the reference to NULL:** possible only when the foreign key is nullable and other constraints permit it.
- **Change to another valid reference:** the replacement must satisfy domains and all relevant constraints; an arbitrary placeholder identifier is insufficient.

In this example, both foreign-key groups in Enrolment are included in its primary key, so **setting them to NULL is not an available repair**. Cascading or changing a reference must also respect business meaning. Do not assume that the database automatically chooses a response.

11. Modification: consider both old and new values

Source: slide 31. The final paragraph clarifies the scope of its statements.

Modified attributes	Checks needed	Example
Primary or other candidate key	New duplicates, NULL in the primary key, and references to the old key	Changing Student S01's sid to S02 creates a key conflict; changing it to S03 instead leaves the old S01 reference without a target unless references are handled
Foreign key	Complete target match, permitted nullability, and any simultaneous primary-key role	Changing courseNo from 101 to 201 in S01's enrolment creates nonexistent `(CS,201)`
Non-key attribute	Domains, mandatory values, and other business constraints still apply	Changing grade to Z violates the allowed-grade rule despite changing no key

Changing S01's enrolment `(dept,courseNo)` together to `(Math,201)` has an existing target and creates no duplicate result key. It is valid under the example's constraints. **Check the entire resulting combination when modifying a composite foreign key.**

The lecture compares changing a primary key to deleting the old key and inserting a new one. This helps remember old references and new conflicts; it does not require physically deleting and reinserting the student.

Clarifying the slides: "Not unless you're modifying the value of a key" and "all relational integrity constraints are to do with the key values" on slide 31 should be read in the context of that slide's key-related constraints. They do not establish that every non-key update is safe: domains, mandatory attributes, and business rules may also be violated.

12. Common distinctions

Pair	Distinction
Relation / ER relationship	Set of tuples / association among entities
Schema / instance	Structure and rules / contents at a particular time
Degree / cardinality	Attribute count / current tuple count
Relation cardinality / ER cardinality ratio	Actual row count / business limits on associations
No duplicate tuples / key uniqueness	No complete-row repetition / no repeated specified key group across distinct rows
Candidate key / primary key	Any minimal superkey / the selected candidate key
Primary key / foreign key	Identifies this relation's tuples / references a target key; attributes may serve both roles
Repeated foreign-key value / dangling reference	Several tuples referencing the same target is usually permitted / referencing a missing target is invalid
Composite-key matching / separate column checks	Match the combination in one target tuple / separate occurrences do not establish validity
Key constraint / entity integrity	Uniqueness / primary-key non-nullability
NULL / zero / empty string	Missing-value marker / a numeric value / a string value
Constraint-valid state / fully correct business state	Obeys declared rules / also depends on whether all necessary business rules have been declared

13. Revision checklist and short-answer language

These are learning goals, not a formal practice test or an official mark allocation.

- ☐ Identify relations, tuples, attributes, domains, schemas, and instances in given tables.
- ☐ State degree and cardinality, including for an empty instance.
- ☐ Explain the difference between rearranging displayed columns and exchanging tuple values.
- ☐ Distinguish no duplicate rows, superkeys, candidate keys, and primary keys.
- ☐ Explain why `2^n − 1` is not an unconditional superkey-count formula.
- ☐ Label the direction of every foreign key and check composite references as combinations.
- ☐ Explain key constraints, entity integrity, and referential integrity separately.
- ☐ Apply the lecture's rules to all-NULL, partly NULL, and nonmatching foreign keys.
- ☐ Assess insertions, deletions, and modifications independently from one initial state, identifying conflicting tuples or missing targets.
- ☐ State the conditions for cascading and setting references to NULL without assuming automatic actions.

Useful short-answer formulations:

A relation is a set of tuples defined over a collection of named attributes.

A relation schema describes the structure; a relation instance contains the tuples at a particular time.

A candidate key is a minimal superkey, and a primary key is a selected candidate key.

Entity integrity requires every component of a primary key to be non-NULL.

Referential integrity requires a non-NULL foreign-key value to match a referenced key value.

A composite foreign key must match the corresponding key components in the same referenced tuple.

Deleting a referenced tuple requires checking the tuples that refer to it.

Sources and preparation record

- Main lecture deck: Lecture2A.pptx, mainly slides 4–32. Text was read and original diagrams/tables on slides 7, 10, 15, 17, 19, 24, 26, 28–29, and 31 were visually inspected, including key underlines, composite references, and operation examples. Mapping steps on slides 33–55 are not developed here.
- Additional comparison: SQLpart1.pptx, slides 31–32, only to distinguish classical set semantics from SQL duplicates; SQL querying is not taught here.
- Scope: CSC5041_Teaching_Plan.pdf, page 1, Week 3. Explanations of degree/cardinality, domain and business constraints, original schemas/data, and limitations of slide wording are distinguished from the original material.
- LaTeX edition prepared on 20 October 2026; original evidence status is retained. No new database execution is claimed.
- Input SHA-256: Lecture2A `cecf28254342cf35e9a0b1b86a007fba728ed17a900e9e05182f2f2616081ee0`; SQLpart1 `aa28c989fb3674e780418c78c483bba20ff5a50b7da067b0341e60410a68c8ed`; syllabus teaching plan `31380a7fe6504e46541b6db0bb2f24859c69188bfeb9681dca73354ebeb68295`.

04 ER-to-Relational Mapping: English Study Notes

CSC5041 · In-class Test 1 revision · 19 October 2026

Scope and learning goals

This chapter connects 02 ER Modelling with 03 Relational Model and Integrity Constraints: translate ER meaning into relation schemas, then check which constraints are preserved and which require additional expression.

The main source is the lecturer's Lecture2A.pptx, slides 33–56, covering "ER-to-relational mapping" in Week 3 on page 1 of the teaching plan. Unless another file is named, slide numbers refer to their order in that deck. Tutorial 1, page 1, Question 1(c)–(d), asks for relational schemas, explanations of which relationships become separate relations, and constraints not enforced by keys and foreign keys alone.

After studying this chapter, you should be able to apply the seven mapping steps and identify primary keys, complete foreign-key groups, relationship-attribute placement, uniqueness, and mandatory references. This chapter covers logical schemas and constraints, not SQL querying or normalization algorithms. Examples restate the lecture or are explicitly original; they are not database execution results or a complete tutorial solution.

1. Mapping inputs, outputs, and notation

Source: slides 33 and 54–56; constraint meanings continue Chapter 03.

```
Input: entities, attributes, keys, relationships, cardinalities, participation, written
↪  rules
  ↓ ER-to-relational mapping
Output: relation schemas + primary keys + foreign-key groups + uniqueness/nullability +
↪  other rules
```

The following notation improves readability; it is **not complete SQL DDL**:

Notation	Meaning
`R(a, b, c)`	Attribute list of relation R
`PK{a,b}`	One composite primary key containing a and b, not two primary keys
`FK{a,b} → S{x,y}`	One composite foreign key, with a corresponding to x and b to y
`UNIQUE{a}`	Non-NULL a values cannot repeat across rows; preserves the relevant uniqueness rule
`NOT NULL{a,b}`	Both a and b must have values

A primary key already requires uniqueness and non-NULL values for every component. Nullable UNIQUE notation here only expresses that existing associations cannot repeat; it does not prescribe how individual SQL products handle NULL. Implementation belongs to Chapter 06.

Tutorial 1(c) requires **underlined primary keys and an asterisk on each foreign-key attribute**. Follow that convention in an actual answer and additionally specify composite-key groups and their targets. Asterisks alone do not identify which columns jointly form one reference.

2. Overview of the seven steps

1:N `Department` 1 — N `Employee`

`Department(`<u>`deptID`</u>`, name)`

`Employee(`<u>`empID`</u>`, name, deptID)`

$Employee.deptID \longrightarrow Department.deptID$

Place FK on the N side; add NOT NULL if Employee participation is total.

M:N `Student` M — N `Course`

`Enrolment(`<u>`sid`</u>`, `<u>`cid`</u>`)` $PK = \{sid, cid\}$

$Enrolment.sid \rightarrow Student.sid$ $Enrolment.cid \rightarrow Course.cid$

Create an association relation; put relationship attributes here. FKs alone do not ensure total participation.

Original simplified example; assume at most one association per student–course pair.

Source: slides 35, 38, 41, 44, 47, 50, and 53–55.

Lecture step	Input	Basic output	Key / constraint concern
1	Strong entity	One relation per strong entity	Retain the selected entity key; flatten composite attributes
2	Weak entity	Relation including the owner's complete key	Owner key plus partial key
3	Binary 1:1 relationship	Usually add one side's key to the other	Foreign-key uniqueness; non-nullability for the carrying side's total participation
4	Binary 1:N relationship	Add the 1-side key to the N-side relation	N-side foreign key; relationship attributes also go there
5	Binary M:N relationship	New relationship relation	Usually the combination of both complete participant keys
6	Multivalued attribute	New attribute relation	Owner key plus all simple components of the attribute value
7	n-ary relationship, n > 2	New relationship relation	Collect participant keys, then determine a minimal identifier from the semantics

Establish complete entity identifiers first, then map regular relationships and multivalued attributes. **Not every diamond requires a new relation**, and not every relationship can be represented by one foreign key. Read uniqueness, nullability, and coverage back against the original model.

Every tuple in a separate relationship relation must have all its participants. Every participating role's key must be non-NULL. Primary-key membership already guarantees this for included components;

explicitly require it for the remaining components. Partial participation is represented by the absence of a relationship row, not by putting a NULL participant into an existing relationship row.

3. Step 1: strong entities and ordinary attributes

Source: slides 35–37; derived attributes continue the ER material on slides 22–23 discussed in Chapter 02.

Create a relation R for strong entity E. Include its simple attributes and the simple components of composite attributes. Use E's selected key as R's primary key. Preserve uniqueness of other candidate keys; selecting one primary key does not remove other identification rules.

Original example: Student has key sid and unique email; composite address(city,street,postcode), multivalued phone, stored dateOfBirth, and derived age.

```
Student(sid, name, email, city, street, postcode, dateOfBirth)
PK{sid}; UNIQUE{email}
```

If email is mandatory in the requirements, also specify non-nullability. This step handles ordinary attributes:

- Flatten address into three attributes rather than mechanically storing a redundant address column as well.
- Do not place phone in a set-valued cell or invent phone1 and phone2; handle it in Step 6.
- Following the lecture's ER example, derive age from dateOfBirth and a reference date rather than automatically storing it again. Materialization would be an additional design choice to justify.
- When an entity key is composite, copy all its components when referencing it elsewhere.

4. Step 2: weak entities and identifying relationships

Source: slides 38–40.

Create a relation for weak entity W containing its simple attributes and the simple components of composite attributes. Include the owner's complete primary key as a foreign key. Its primary key combines **the owner key and W's partial key**.

```
Employee(empID, name)                          PK{empID}
Dependent(empID, dependentName, dateOfBirth, kinship)
  PK{empID, dependentName}
  FK{empID} → Employee{empID}
```

This restates the lecture's Dependent example using consistent names. dependentName is unique only within one Employee's dependents, so it cannot be the sole primary key. empID is both a primary-key component and a foreign key; its non-nullability reflects the weak entity's required owner.

The identifying relationship usually needs no separate relation repeating these keys. If it has attributes, they can accompany the reference in the weak entity's relation under this example's exactly-one-owner condition.

When the owner is itself weak

Original example: roomNo identifies a Room within a Building; deskNo identifies a Desk within a Room.

```
Building(buildingID, name)                  PK{buildingID}
Room(buildingID, roomNo, floor)
  PK{buildingID, roomNo}; FK{buildingID} → Building{buildingID}
Desk(buildingID, roomNo, deskNo)
  PK{buildingID, roomNo, deskNo}
  FK{buildingID,roomNo} → Room{buildingID,roomNo}
```

Desk references Room through a two-attribute foreign key. Copying only roomNo loses the building context. Do not introduce a new global ID merely because the object is weak and then discard the original local-uniqueness requirements.

5. Step 3: binary 1:1 relationships

Source: slides 41–43. Explicit uniqueness and coverage checks below supplement the mapping semantics.

Let S and T represent the two entity types. Usually add the complete primary key of one side to the other as a foreign key, together with relationship attributes. The lecture recommends carrying the foreign key on the totally participating side when one exists.

Original example: each Department must have exactly one Employee as manager; each Employee manages at most one Department. MANAGES has attribute startDate.

```
Employee(empID, name)                       PK{empID}
Department(deptID, name, managerID, managerStartDate)
  PK{deptID}
  FK{managerID} → Employee{empID}
  UNIQUE{managerID}; NOT NULL{managerID}
```

Meaning to preserve	Rule expressing it
The manager is an existing Employee	Foreign key
Every Department has exactly one manager	Single-valued managerID plus non-nullability and the foreign key
Each Employee manages at most one Department	Uniqueness of managerID
The starting date describes this management association	Store it in Department alongside the management reference

A foreign key without UNIQUE permits several departments to select the same employee, producing N:1 instead of 1:1. Slide 41 does not list every constraint declaration; that does not justify dropping the 1:1 semantics. Whether a relationship attribute is mandatory is a separate requirement, not an automatic consequence of total participation.

Both sides partial, or a separate relationship relation

One option is a nullable, unique foreign key on one side. Another is:

```
Manages(deptID, empID, startDate)
PK{deptID}; UNIQUE{empID}; NOT NULL{empID}
FK{deptID} → Department{deptID}; FK{empID} → Employee{empID}
```

This permits at most one manager per department and at most one managed department per employee. Non-NULL empID ensures that every existing management association has an employee participant. It **does not require every Department to appear in Manages**. A totally participating side needs an additional coverage requirement when this separate-relation representation is used.

The lecture also mentions merging both types where appropriate, especially when both participate totally and have no other relationships. Do not merge mechanically whenever a relationship is 1:1. Check whether identifiers, independent meanings, and other associations remain correctly represented.

6. Step 4: binary 1:N relationships

Source: slides 44–46.

Add the **complete key of the 1 side to the N-side relation as a foreign key**. Place relationship attributes on the N side too. The original N-side primary key usually stays unchanged.

Original example: Department:Employee = 1:N. Each Employee must work in one Department; WORKS_FOR has attribute joinedOn.

```
Department(deptID, name)                    PK{deptID}
Employee(empID, name, deptID, joinedOn)
  PK{empID}; FK{deptID} → Department{deptID}
  NOT NULL{deptID}
```

- Employee.deptID can repeat because a department can have several employees. Adding UNIQUE would incorrectly restrict the association to 1:1.
- joinedOn describes the employee's association with that department and accompanies the foreign key in Employee.
- Employee's total participation requires non-NULL deptID. If employees may have no department, allow a null reference and explain how relationship attributes are handled when no association exists.
- If every Department must have an Employee, Employee.deptID being a non-NULL foreign key still does not guarantee that requirement. A Department could remain unreferenced.

Do not store a list of employee IDs in one Department cell. The course's flat relational model does not directly store such multivalued sets as attributes.

7. Step 5: binary M:N relationships

Source: slides 47–49.

Create a relationship relation containing both complete participant keys as foreign keys, together with relationship attributes. Under ordinary ER semantics of one relationship instance per participant pair, use their combined keys as the primary key.

```
Employee(empID, name)                       PK{empID}
```

```
Project(projectID, title)                PK{projectID}
WorksOn(empID, projectID, timeFraction)
  PK{empID, projectID}
  FK{empID} → Employee{empID}
  FK{projectID} → Project{projectID}
```

empID alone would allow at most one project per employee; projectID alone would allow at most one employee per project. timeFraction describes a pair's property and normally does not belong in the key. Including it would permit repeated pairs with different fractions.

Adding one foreign key to each entity relation also cannot represent arbitrary M:N pairings. A relationship relation uses separate rows for the pairings without imposing a single-valued association on either participant.

What if the same pair can participate repeatedly?

If repeated borrowings of the same Book by the same Reader must be stored, explicitly model a Loan event and its identification. An original extension is `Loan(loanID, readerID, bookID, startDate, endDate)`, with primary key loanID, `FK{readerID}` → `Reader{readerID}`, and `FK{bookID}` → `Book{bookID}`. Require `NOT NULL{readerID,bookID}` so that each loan has both a reader and a book.

A new event identifier needs a requirement-based justification. Do not arbitrarily add every relationship attribute to the key just to allow repeats. Ordinary M:N mapping and repeated-event modelling have different assumptions.

8. Step 6: multivalued attributes

Source: slides 50–52.

Create a separate relation containing the owner's complete key as a foreign key and the attribute value's simple components. Under the lecture's set semantics, and without an additional smaller identification rule, **owner key plus attribute value** forms the primary key.

A multivalued simple attribute

```
Department(deptID, name)                  PK{deptID}
DepartmentPhone(deptID, phone)
  PK{deptID, phone}; FK{deptID} → Department{deptID}
```

Using only deptID permits at most one phone per department. Using only phone additionally prohibits departments from sharing a number unless that is actually required. A department with no phone has no corresponding DepartmentPhone row, rather than a `(deptID,NULL)` row: phone is part of the primary key here.

A multivalued composite attribute

Original example: a Person can have several Addresses, each consisting of city, street, and postcode.

```
PersonAddress(personID, city, street, postcode)
PK{personID, city, street, postcode}
FK{personID} → Person{personID}
```

The complete address value distinguishes addresses for the same person. This assumes the identifying components are available in full and no additional smaller identifier is specified. The same address cannot repeat for one person, matching the attribute's set semantics. Incomplete addresses, address IDs, labels, or independent history require explicit additional requirements before changing this design.

Slide naming clarification: slide 34 labels the multivalued attribute Phone, whereas slide 52 names the final relation D_number with Dnumber. This chapter consistently uses DepartmentPhone/phone to explain the mapping rather than treating that naming change as a newly required property.

9. Step 7: n-ary relationships and key selection

Source: slide 53. The ternary examples below explain the abbreviated key-selection rule; they are not original lecture questions.

Create a relation for an n-ary relationship, collect each participating role's complete entity key as a foreign key, and add relationship attributes. **Selecting a key is not circling every column or simply looking for the entity labelled 1.**

A ternary relationship without additional uniqueness restrictions

```
Supply(supplierID, partID, projectID, quantity)
PK{supplierID, partID, projectID}
FK{supplierID} → Supplier{supplierID}
FK{partID} → Part{partID}
FK{projectID} → Project{projectID}
```

Assume at most one supply association per triple and that no pair of participants determines the third. The three participant keys form the primary key; quantity is a relationship attribute.

Suppose a fixed Supplier and Part have at most one Project

Then `(supplierID,partID)` already determines the supply association. Assuming no smaller unique group, choose `PK{supplierID,partID}`. projectID remains a foreign key but need not be part of the primary key. Explicitly add `NOT NULL{projectID}` so that every ternary association still has a Project participant; removing it from the primary key does not make that role optional within a relationship tuple.

For example, `(S1,P1,J1,10)` and `(S1,P1,J2,20)` have different three-part keys. Using only the full triple as the key would not prevent one Supplier–Part pair from having two Projects. **Preserving the at-most-one rule requires uniqueness of the participant group that determines that project.**

The exception wording on slide 53 does not spell out which roles are fixed before "at most one" is assessed. This chapter does not infer that the key of the side labelled 1 must alone be the primary key. Check minimal identifiers against explicit uniqueness rules and preserve other candidate-key uniqueness where several candidates exist.

Likewise, do not automatically replace a ternary relationship with three binary ones: that can lose information about which three objects participate together. If the requirements are ambiguous, state which association meaning needs confirmation.

10. Worked example: the lecture's company model

Source: slides 34–52. The following retains the entities and associations but uses consistent names: SSN → empID, Department.Name → deptName, Pnumber → projectID, and Dependent.Fname → dependentName. Department still uses its name as its key, as in the lecture; **no new deptID is invented**. This is a given assumption of this example, not a claim that all real department names are unique.

Input meaning

- Strong entities: Employee, Department, and Project. Weak entity Dependent is identified through Employee and dependentName.
- WORKS_FOR: each Employee works in exactly one Department; a Department can have several employees.
- MANAGES: each Department has exactly one manager; an Employee manages at most one Department. The relationship has attribute managerStartDate.
- WORKS_ON: Employee and Project have an M:N relationship with attribute timeFraction.
- Department has multivalued phone; Dependent also has dateOfBirth and kinship.

Final schemas and keys

Relation schema	Primary key	Foreign keys
Employee(empID, firstName, lastName, dateOfBirth, deptName)	`{empID}`	`{deptName}` → `Department{deptName}`
Department(deptName, location, managerID, managerStartDate)	`{deptName}`	`{managerID}` → `Employee{empID}`
Project(projectID, projectName)	`{projectID}`	None
Dependent(empID, dependentName, dateOfBirth, kinship)	`{empID,dependentName}`	`{empID}` → `Employee{empID}`
WorksOn(empID, projectID, timeFraction)	`{empID,projectID}`	`{empID}` → `Employee{empID}`; `{projectID}` → `Project{projectID}`
DepartmentPhone(deptName, phone)	`{deptName,phone}`	`{deptName}` → `Department{deptName}`

Also specify `UNIQUE{Department.managerID}`, `NOT NULL{Department.managerID}`, and `NOT NULL{Employee.deptName}`. Every primary key is already non-NULL. Decide nullability of other ordinary attributes from the requirements rather than guessing from their names.

Where did each ER association go?

ER structure	Mapping destination	Separate relationship relation?
Identifying relationship between Employee and Dependent	Dependent.empID and its composite primary key	Usually no
MANAGES, 1:1	Department.managerID and managerStartDate, with uniqueness	No
WORKS_FOR, Department:Employee = 1:N	Employee.deptName	No
WORKS_ON, M:N	WorksOn	Yes

ER structure	Mapping destination	Separate relationship relation?
Department's multivalued phone	DepartmentPhone	Yes as an attribute relation, not a newly introduced ER binary relationship

Mutual references between Employee and Department are not automatically a design error: they represent different associations. They also **do not automatically ensure that a manager works in the department managed**. If that rule is additionally required, express it explicitly as discussed next.

11. Constraints to check after mapping

Source: reading back the semantics of slides 41, 44, 47, and 55; application context from Tutorial 1, Question 1(d). Mechanisms below are explanatory supplements, not implemented database features.

Business rule	Do ordinary primary/foreign keys already suffice?	Additional expression needed
An Employee manages at most one Department in a 1:1 relationship	No: a managerID foreign key permits repetition	Uniqueness of managerID
Every tuple on the carrying side must have a target	A nullable foreign key is insufficient	Foreign-key non-nullability
Every Department has at least one Employee	Employee's foreign key remains insufficient even if non-NULL	Global coverage: every department is referenced by an employee
Every Project participates in an M:N association	WorksOn's primary key and two foreign keys are insufficient	Every Project occurs in at least one WorksOn tuple
A manager works in the department managed	Keys for the two separate associations are insufficient	A cross-association rule requiring management pairs to be employment pairs
startDate is no later than endDate	No	A constraint comparing values

Uniqueness and mandatory values can be expressed through corresponding declarations; date ordering can use a check constraint. Global coverage needs a suitable global check, trigger, or similar mechanism. The current foreign keys cannot be presented as already guaranteeing it.

One precise expression of the same-department manager rule: in Section 10, Department's `(managerID,deptName)` must occur in Employee's `(empID,deptName)`. An implementation supporting the corresponding composite-key references can declare that Employee attribute group unique and add this composite foreign key. Alternatively, triggers or similar mechanisms must correctly cover the relevant changes.

Thus "the current single-column foreign keys are insufficient" differs from "no redesigned schema could ever express this through a composite foreign key." An answer to Tutorial 1(d) should specify its schema and mechanism rather than make unconditional impossibility claims about keys and foreign keys.

12. A practical answer-writing procedure

Source: slides 54–56; the sequence below organizes the material into a workflow.

1. **Record input constraints.** List entity keys, weak-entity owners, minima and maxima for every relationship, and extra written rules.

2. **Create entity relations first.** Flatten ordinary attributes of strong entities; give weak entities complete owner keys and partial keys.
3. **Map regular relationships.** Choose a carrying side and uniqueness for 1:1; use the N side for 1:N; create separate M:N and n-ary relations.
4. **Place relationship and multivalued attributes.** Store the former with their associations and the latter in separate relations. Include simple components of composite attributes.
5. **List primary keys and every foreign-key group.** State the target attribute group rather than marking an unexplained "FK."
6. **Read maxima and minima back.** Did 1:N accidentally become 1:1? Are uniqueness, non-nullability, or total-participation coverage missing?
7. **Try counterexamples.** Can two departments share one manager? Can a department exist without employees? If forbidden by the requirements but permitted by the schema, another constraint is needed.
8. **State mechanisms and assumptions.** Separate schema guarantees from checks still requiring implementation. Follow requested underlining and asterisk conventions.

These are static schema checks, not SQL execution verification. Listing relation names alone does not establish preservation of all ER semantics.

13. Common distinctions

Pair	Distinction
Strong-entity key / weak-entity partial key	Complete independent identifier / identification requiring owner context
Composite foreign key / several independent foreign keys	One group referencing one target key / independently targeted references
Foreign key / uniqueness in 1:1	Target existence / preventing several source tuples from selecting the same target
N-side foreign key / list of IDs on the 1 side	One reference per tuple / incorrectly storing a multivalued set in one attribute
Non-NULL foreign key / target-side total participation	Every referencing tuple has a target / every target has a referencing tuple
Relationship attribute / identifying attribute	Property of an association / an identification role requiring a separate semantic justification
M:N relation / multivalued-attribute relation	Participant pairings / multiple attribute values of one owner
Complete triple of keys / ternary candidate key	Distinguishes triples / may not be minimal under stronger uniqueness rules
Foreign-key arrows / complete constraint preservation	Reference directions / uniqueness, nullability, coverage, and remaining rules must also be checked

14. Revision checklist and short-answer language

These are learning goals, not a formal practice paper or an official mark allocation.

☐ State the seven mapping steps and their outputs without consulting the notes.
☐ Flatten composite attributes and handle multivalued and derived attributes separately.
☐ Write complete primary keys and grouped foreign keys for two levels of weak entities.
☐ Explain why a 1:1 foreign key normally also needs uniqueness.

- ☐ Place foreign keys and relationship attributes correctly for 1:N.
- ☐ Select a combined key for an ordinary M:N relation without arbitrarily adding relationship attributes.
- ☐ Map simple and composite multivalued attributes and explain how no value is represented.
- ☐ Determine a minimal identifier from explicit ternary uniqueness rules rather than the location of a 1 label.
- ☐ Read every key, reference, uniqueness rule, and mandatory value back in the worked example.
- ☐ Identify total-participation or cross-association rules not yet guaranteed by the foreign keys and describe suitable mechanisms.

Useful short-answer formulations:

> Map a strong entity type to a relation containing its simple attributes and the simple components of composite attributes.
>
> The primary key of a weak entity's relation combines the owner's primary key with the weak entity's partial key.
>
> For a binary 1:N relationship, place the foreign key and relationship attributes in the relation on the N side.
>
> A foreign key alone does not preserve a 1:1 relationship; uniqueness must also be enforced.
>
> Map a many-to-many relationship to a separate relation containing the keys of both participants and the relationship attributes.
>
> A non-NULL foreign key ensures that each referencing tuple has a target, not that every target is referenced.
>
> For an n-ary relationship, choose the key from the stated uniqueness constraints rather than from the diagram labels alone.

Sources and preparation record

- Main lecture deck: Lecture2A.pptx, slides 33–56. Text was read and original diagrams on slides 34, 37, 40, 43, 46, 49–50, 52–53, and 55 were visually inspected for key underlines, foreign-key arrows, participation lines, and multivalued-attribute naming.
- Scope: CSC5041_Teaching_Plan.pdf, page 1, Week 3, and page 2, Tutorial 1. Supporting requirements: Tutorial1.pdf, page 1, Question 1(c)–(d). This chapter supplies neither a complete clinic solution nor a practice paper.
- The Step 1 explanation of derived attributes continues ER slides 22–23 as discussed in Chapter 02. Original examples, consistent renaming, explicit constraint supplements, and clarifications of abbreviated slide rules are identified where used.
- LaTeX edition prepared on 20 October 2026; original evidence status is retained. No new database execution is claimed.
- Input SHA-256: Lecture2A `cecf28254342cf35e9a0b1b86a007fba728ed17a900e9e05182f2f2616081ee0`; Tutorial 1 `316e8b6bb9581a5ecae3fb2f449e395a4ea8a19b4ee16c4191fc20a6079d2c44`; syllabus teaching plan `31380a7fe6504e46541b6db0bb2f24859c69188bfeb9681dca73354ebeb68295`.

05 Relational Algebra: English Study Notes

CSC5041 · In-class Test 1 revision · 19 October 2026

Scope and learning goals

This chapter follows 03 Relational Model and 04 ER Mapping. The main source is the lecturer's Relational_Algebra.pptx, 61 slides. Unless another file is named, slide numbers refer to their order in that deck.

The scope corresponds to Week 4 on page 1 of the teaching plan. Tutorial 2, page 1, Question 1(a)(i)–(vi), covers selection/projection, joins, set operations, students taking no courses, every target course, and grouped aggregation. Aggregation is not part of the classical basic algebra, but both slides 58–59 and the tutorial cover it, so it is included here. SQL belongs to Chapter 06.

After studying this chapter, you should be able to explain input and result schemas, compose expressions from written requirements, and detect incorrect joins, premature projection, name collisions, and division candidate-scope problems. Original data and results are static teaching deductions, not real data, database execution results, or a new practice paper.

1. Notation and closure

Source: slides 2–4 and 55–61.

Relational algebra takes relations as inputs and produces relations that can be used in further operations. This is **closure**. The lecture calls it a procedural DML: an expression specifies a composition of logical operations, not an obligatory physical execution sequence for the database engine.

Operation	Notation used here	Main purpose
Selection	$\sigma_{\text{condition}}(R)$	Filter rows
Projection	$\pi_{\text{attributes}}(R)$	Retain columns and remove duplicates
Rename	$\rho_{\text{NewName(newAttributes)}}(R)$	Distinguish roles or align attribute names
Union, intersection, difference	$R \cup S$, $R \cap S$, $\texttt{R} - \texttt{S}$	Either, both, or left but not right
Cartesian product	$\texttt{R} \times \texttt{S}$	All tuple pairings
Theta join	$R \bowtie_{\text{condition}} S$	Pairings satisfying a condition
Natural join	$R \bowtie S$	Match all common attributes and merge them
Division	$R \div S$	Candidates associated with every target in S
Grouping and aggregation	$\gamma_{\text{groups; aggregates}}(R)$	Statistics per group

Braces `{...}` in code blocks represent subscripts for stable Markdown display. ← names an intermediate relation without updating the stored data. Renaming changes relation/attribute identifiers, not existing values; a full attribute-renaming list corresponds positionally to the original schema.

Examples use set semantics and contain no NULL values. Projection eliminates duplicates, and there is no implicit row order. Do not import SQL duplicate handling, NULL, or three-valued logic into these classical operations without defining the extension. NULL in the tutorial's grade does not affect expres-

sions using only other attributes; NULL query semantics belong to Chapter 06.

2. Running example: specify the inputs first

This is an original simplified enrolment database, **not the tutorial schema**. It omits term; Takes records whether a student has taken a course.

```
Student(sid, sname, major)          PK{sid}
Course(cid, title, dept, credits)   PK{cid}
Takes(sid, cid)                    PK{sid,cid}
Takes.sid → Student.sid; Takes.cid → Course.cid
```

Student:

sid	sname	major
S1	Amy	CS
S2	Ben	Math
S3	Amy	CS
S4	Dana	Math

Course:

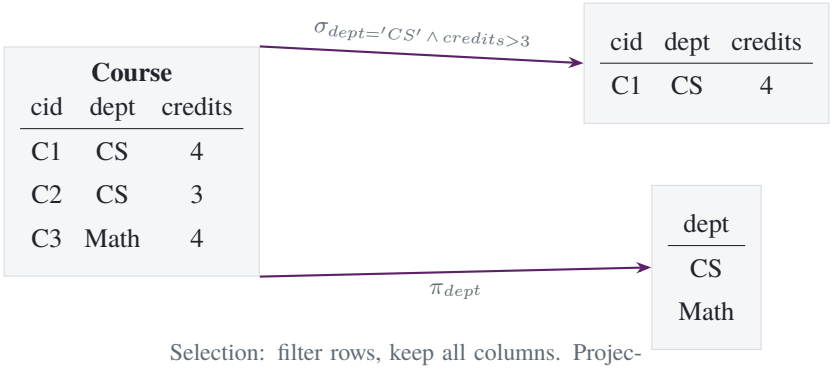
cid	title	dept	credits
C1	Databases	CS	4
C2	AI	CS	3
C3	Linear Algebra	Math	4

Takes:

sid	cid
S1	C1
S1	C2
S2	C1
S2	C3
S3	C3

S1 and S3 share a name: **names are not identifiers**. S3 majors in CS but has taken only a Math course: **a student's major and a course's department are different conditions**. S4 has no enrolments and must remain available when considering "never" and missing zero-count groups.

3. Selection σ: filter tuples, retain all attributes



Selection: filter rows, keep all columns. Projection: retain columns and eliminate duplicates.
Chapter 05 data; the title column is omitted for display.

Source: slides 5–10 and 18.

$\sigma_{\text{dept='CS'} \wedge \text{credits}>3}(\text{Course})$

The result schema remains `(cid,title,dept,credits)` and contains only `(C1,Databases,CS,4)` here. Selection does not remove columns or change retained values. Conditions can compare an attribute to a constant or another attribute and combine predicates with $\wedge$, $\vee$, and $\neg$.

Consecutive selections can be combined and reordered:

$$\sigma_p(\sigma_q(R)) = \sigma_{p \wedge q}(R) = \sigma_q(\sigma_p(R))$$

The conditions must be defined over R's attributes. "CS courses with more than 3 credits" can use consecutive selections; it differs from "CS courses or courses with more than 3 credits."

Do not confuse SQL SELECT with algebraic selection: σ filters rows; the SQL SELECT list mainly specifies output columns.

4. Projection π: retain attributes and eliminate duplicates

Source: slides 11–18.

$\pi_{\text{dept}}(\text{Course}) = \{(\text{CS}), (\text{Math})\}$

Two CS courses still produce only one CS tuple because the result is a set. The result schema contains only dept; cid, title, and credits are no longer available.

Combine selection and projection:

$\pi_{\text{title}}(\sigma_{\text{dept='CS'} \wedge \text{credits}>3}(\text{Course}))$

Its schema is `(title)` and its result is `{(Databases)}`. Read from the innermost parentheses outward: filter courses, then retain titles.

Do not discard attributes needed later

Valid : $\pi_{\text{title}}(\sigma_{\text{dept='CS'}}(\text{Course}))$

Invalid : $\sigma_{\text{dept='CS'}}(\pi_{\text{title}}(\text{Course}))$

The second expression projects dept away before trying to use it. In general, the following requires all attributes used by p to remain in X:

$\pi_X(\sigma_p(R)) = \sigma_p(\pi_X(R))$

Nested projections satisfy $\pi_X(\pi_Y(R)) = \pi_X(R)$ when $X \subseteq Y$. Requesting an attribute already removed by the inner projection makes the expression undefined, not an empty result.

5. Rename ρ: distinguish roles and conflicting names

Source: slides 55–56 and 60.

Renaming aligns schemas for set operations, distinguishes roles in self-joins, and prevents attribute conflicts in products and conditional joins. For pairs of distinct students with the same major:

$A \leftarrow \rho_{A(\mathsf{aid,aname,amajor})}(\mathsf{Student})$

$B \leftarrow \rho_{B(\mathsf{bid,bname,bmajor})}(\mathsf{Student})$

$\mathsf{Pairs} \leftarrow \pi_{\mathsf{aid,bid}}(A \bowtie_{\mathsf{amajor=bmajor} \wedge \mathsf{aid<bid}} B)$

Assume ID order `S1<S2<S3<S4`. The result is `{(S1,S3),(S2,S4)}`. `aid<bid` excludes self-pairings and reverse duplicates. Using only $\mathsf{aid} \neq \mathsf{bid}$ returns both `(S1,S3)` and `(S3,S1)`.

Renaming does not change Amy's actual name. A natural join of Student with an unrenamed copy of itself does not enumerate different-student pairs: it matches equality on all common attributes.

6. Union, intersection, and difference: align schemas first

Source: slides 19–32 and 56.

Operation	Tuples retained	Commutative?
$R \cup S$	In R or S, including those in both	Yes
$R \cap S$	In both R and S	Yes
`R − S`	In R but not S	Generally no

All three require **union compatibility**. Slide 19 requires a one-to-one correspondence of attributes with the same names and domains. Project matching attribute groups and rename/align them if necessary. Equal column counts alone do not make relations compatible.

Build two **student-ID sets** in the running example:

$\mathsf{CSIDs} \leftarrow \pi_{\mathsf{sid}}(\mathsf{Takes} \bowtie \sigma_{\mathsf{dept='CS'}}(\mathsf{Course}))$

$\mathsf{MathIDs} \leftarrow \pi_{\mathsf{sid}}(\mathsf{Takes} \bowtie \sigma_{\mathsf{dept='Math'}}(\mathsf{Course}))$

Then `CSIDs={S1,S2}` and `MathIDs={S2,S3}`. The shorthand below omits parentheses around individual one-attribute tuples.

Expression	Result here	Meaning
$\mathsf{CSIDs} \cup \mathsf{MathIDs}$	`{S1,S2,S3}`	At least one course from either department
$\mathsf{CSIDs} \cap \mathsf{MathIDs}$	`{S2}`	Courses from both departments
`CSIDs − MathIDs`	`{S1}`	CS courses but no Math courses
`MathIDs − CSIDs`	`{S3}`	Math courses but no CS courses

"Or" here is inclusive. "Exactly one category" needs an additional exclusion, for example $(\mathsf{CSIDs} -$

$\mathsf{MathIDs}) \cup (\mathsf{MathIDs} - \mathsf{CSIDs})$.

"Never enrolled" is not an inequality on one record

$\mathsf{NeverIDs} \leftarrow \pi_{\mathsf{sid}}(\mathsf{Student}) - \pi_{\mathsf{sid}}(\mathsf{Takes})$

The result is `{S4}`. Start with all candidate students, then subtract those with at least one enrolment. A row satisfying $\mathsf{cid} \neq \mathsf{C1}$ does not show that its student never took C1: the same student may have taken both C1 and C3.

7. Products and joins: all pairings versus correct pairings

Source: slides 33–47 and 61.

Cartesian product $\times$

Pair each R tuple with each S tuple. If R has m rows and S has n, the product has $m \times n$ rows. After disambiguating attribute names, its degree is the sum of the input degrees. Student$\times$Course has $4 \times 3 = 12$ rows and seven attributes here.

A product does not automatically match primary and foreign keys. Rename conflicting attributes first. In conditional joins, this chapter also uses `relation.attribute` to identify origins; this can be understood as correspondingly qualifying/renaming conflicting attributes.

Theta and equi-joins

$R \bowtie_\theta S = \sigma_\theta(R \times S)$

The identity assumes disambiguated attributes and a well-defined condition. A theta condition can compare values; a join using only equality comparisons is an equi-join.

$\mathsf{Takes} \bowtie_{\mathsf{Takes.cid}=\mathsf{Course.cid}} \mathsf{Course}$

Only matching course IDs are paired, producing five rows here. An ordinary equi-join retains both copies of the join attributes, distinguished by qualified names. **Equal values do not automatically merge the two columns.**

Natural join $\bowtie$

A natural join equates **all identically named attributes** and retains one copy of each. With no common attribute names, it reduces to a Cartesian product.

$\mathsf{Student} \bowtie \mathsf{Takes} \bowtie \mathsf{Course}$

Here the first join shares only sid, and the next shares only cid. The result schema is `(sid,sname,major, cid,title,dept,credits)`, with five tuples.

A natural join does not infer which columns should match semantically. If Instructor and Course both contain dept, it can impose equality between instructor and course departments. If that is not required, use explicit join conditions or rename unrelated common attributes first.

Slide 37 connects equality filtering followed by removing a duplicate join column with natural joining. Apply the all-common-attributes rule on slides 40–41: not every equi-join followed by projection is automatically a natural join. Joins retain matching combinations; they do not automatically add a row for unenrolled S4.

8. Composing queries: preserve identity, project names last

Source: slides 25–32 and 43–47. The shared-name counterexample is original.

To find names of students who have taken both CS and Math courses, intersect IDs and then join Student:

$$\mathsf{BothIDs} \leftarrow \mathsf{CSIDs} \cap \mathsf{MathIDs}$$

$$\mathsf{Answer} \leftarrow \pi_{\mathsf{sname}}(\mathsf{Student} \bowtie \mathsf{BothIDs})$$

This returns `{(Ben)}`. If names are projected first, both sets contain Amy: S1 supplies the CS occurrence and S3 supplies the Math occurrence. Intersecting names wrongly returns Amy by combining **two different people** into one person satisfying both conditions.

For "CS but no Math," also subtract IDs first. The correct student S1 is named Amy, but subtracting the name sets removes Amy. **Projection distributes over union, but cannot generally be moved ahead of intersection or difference.**

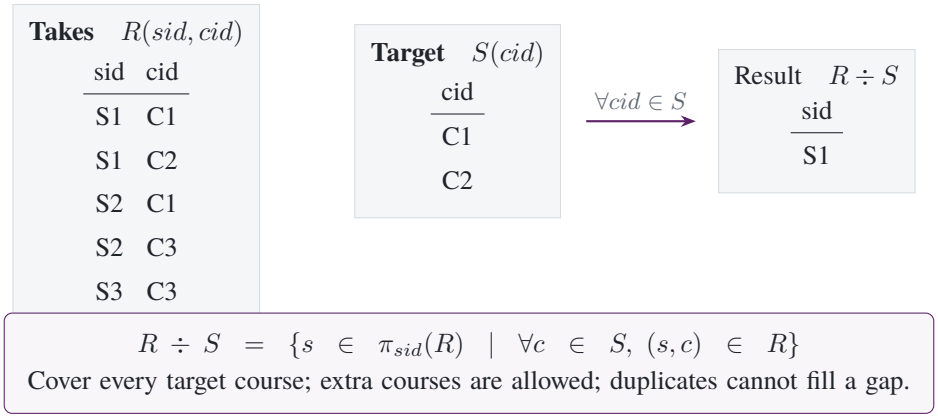
If only names are requested, the final projection merges namesakes as a name set. Retain sid when students must remain distinguishable. Requested name output does not make names keys.

Do not drop the tutorial's term prematurely

Tutorial 2's Enrolled and Teaches both include cid and term. Taking a course taught by an instructor requires matching the course and its offering term. Matching only cid can incorrectly associate an enrolment from another term with that instructor.

Takes deliberately omits term here and is not a direct replacement for the tutorial's complete schema. Check the full linking attribute group before reusing a template.

9. Division ÷: express "every target is covered"



Chapter 05 data. S2 lacks C2; S3 lacks C1 and C2.
Candidates come from $\pi_{sid}(R)$, so S4 is outside this division's candidate domain.

Source: slides 48–54 and 61.

Let R have disjoint attribute groups X and Y, and let S have exactly attributes Y. $R \div S$ returns X values paired in R with **every tuple of S**, taking candidates from $\pi_X(R)$.

$$R(X,Y) \div S(Y) \rightarrow \mathsf{Result}(X)$$

This is not numeric division or matching any one target. Identify the candidate group X and the required target group Y first.

Students who have taken every CS course

$R \leftarrow \pi_{\text{sid,cid}}(\text{Takes})$

$D \leftarrow \pi_{\text{cid}}(\sigma_{\text{dept='cs'}}(\text{Course}))$

$\text{AllCSIDs} \leftarrow R \div D$

D contains `{C1,C2}`. S1 covers both; S2 covers only C1; S3 covers only C3. The result is `{S1}`. Names can then be returned with $\pi_{\text{sname}}(\text{Student} \bowtie \text{AllCSIDs})$.

- Take "all CS courses" from the complete Course relation, not only existing enrolments, or an unchosen required course may disappear from the target set.
- Taking every CS course permits additional Math courses. It does not mean taking only CS courses.
- Project the dividend to `(sid,cid)`. Extra sname, term, or grade attributes would remain in the quotient and change which fixed candidates must cover all targets.

Multiple target attributes and result schemas

Slides 53–54 use R(A,B,C) and S(B,C):

$R \div S \quad \rightarrow$ returns A; covers every (B, C) pair in S

$R \div \pi_C(S) \quad \rightarrow$ returns (A, B); each fixed (A, B) covers every C

These ask different questions. For the given slide instance, the first returns `{a1}`, while the second returns `{(a1,b1),(a3,b2)}`. Seeing "all" does not remove the need to inspect the target schema.

10. Expanding division with difference, and empty targets

Source: slide 57. Candidate-universe and empty-target discussion clarifies the definition's boundaries.

Rewrite "every target is present" as "no required target is missing":

$\text{Candidates} \leftarrow \pi_X(R)$

$\text{Required} \leftarrow \text{Candidates} \times S$

$\text{Missing} \leftarrow \text{Required} - R$

$\text{Bad} \leftarrow \pi_X(\text{Missing})$

$\text{Result} \leftarrow \text{Candidates} - \text{Bad}$

R has exactly `(X,Y)` here; align the schemas of Required and R. Project wider inputs to X and Y first. X and Y denote attribute groups, not a set difference between relation names.

In the running example, Candidates contains `{S1,S2,S3}`. Required contains six student–course pairs. Missing contains `(S2,C2)`, `(S3,C1)`, and `(S3,C2)`, so Bad is `{S2,S3}` and only S1 remains. Additional Math pairs in R do not create missing pairs and therefore do not disqualify S1.

Empty targets require an explicit candidate scope

If S is empty, Required and Missing are empty, and standard division returns $\pi_X(R)$. It **does not automatically include students who never occur in R**.

If all Student tuples should be candidates for a universal condition, use an explicit universe U:

$U \leftarrow \pi_{\text{sid}}(\text{Student})$

$\text{Bad} \leftarrow \pi_{\text{sid}}((U \times D) - R)$

$\mathsf{Result} \leftarrow U - \mathsf{Bad}$

R and D are from Section 9. For nonempty D this agrees with standard division under the running example's valid foreign keys. For empty D it returns all U, including unenrolled S4. This is the truth of a universal condition over an empty target set, not a guarantee that every practical question allows an empty target. State assumptions if the question excludes that case.

11. γ: grouping and aggregation

Source: slides 58–59 and Tutorial 2, Question 1(a)(vi). The notation below is the convention used in these notes.

γ supports COUNT, SUM, AVG, MIN, and MAX. $\gamma_{\mathsf{groupAttributes};\, \mathsf{aggregate}\to\mathsf{resultName}}(R)$ produces one tuple per group, retaining group attributes and adding summary values.

$\mathsf{Stats} \leftarrow \gamma_{\mathsf{dept};\, \mathrm{COUNT}(\mathsf{cid})\to\mathsf{nCourses},\, \mathrm{AVG}(\mathsf{credits})\to\mathsf{avgCredits}}(\mathsf{Course})$

dept	nCourses	avgCredits
CS	2	3.5
Math	1	4

Filter groups with more than one course using $\sigma_{\mathsf{nCourses}>1}(\mathsf{Stats})$; only CS remains. Define an aggregate result name in γ before using it in an outer selection.

What object is being counted?

- Course has one tuple per course and unique cid, so COUNT(cid) counts courses here.
- Joining Takes first can give several tuples for one course, one per student. Counting directly may then count enrolment pairings rather than distinct courses.
- To count distinct objects, deduplicate at the correct identifier level. For example, project a joined result to `(dept,cid,credits)` before grouping. Projecting only `(dept,credits)` can merge different courses with equal credits.
- $\gamma_{\mathsf{sid};\, \mathrm{COUNT}(\mathsf{cid})\to\mathsf{nTaken}}(\mathsf{Takes})$ produces groups for S1, S2, and S3 with counts 2, 2, and 1. It does not automatically generate S4 with a zero count.

All groups aggregated here are nonempty and contain no NULL. Slide 58 raises NULL but does not fully specify its semantics there. Empty-input and NULL handling require explicit conventions; SQL COUNT and NULL details belong to Chapter 06.

12. Equivalent rewrites: check the conditions first

Source: slides 10, 15–17, 57, and 60. Conditions below explain the relevant definitions.

Rewrite	Condition or meaning
$\sigma_p(\sigma_q(R)) = \sigma_{p\wedge q}(R)$	Both predicates are defined
$\pi_X(\pi_Y(R)) = \pi_X(R)$	$X \subseteq Y$
$\pi_X(\sigma_p(R)) = \sigma_p(\pi_X(R))$	X retains every attribute used by p
$R \cap S = R - (R - S)$	R and S are union-compatible
$R \bowtie_\theta S = \sigma_\theta(R \times S)$	Attributes are disambiguated and θ is defined

Rewrite	Condition or meaning
$\pi_X(R \cup S) = \pi_X(R) \cup \pi_X(S)$	Compatible inputs, available X attributes, and set semantics

Replacing $\cup$ by $\cap$ or $-$ in the last rule is not generally valid: projection can discard identity, as the namesake example in Section 8 shows. Projection pushed before a join must also retain output attributes, join attributes, and attributes needed for later filters.

Slide 57 lists basic operators σ, π, $\cup$, $-$, $\times$, ρ; joins, intersection, and division can be derived from them. Aggregation on slide 58 adds statistical power. **General counting is not simply derived from these classical basic operators.** Distinguish derived operations from aggregation that extends expressive power.

13. A procedure for translating English requirements

Source: slides 25–32, 43–54, and 60–61; the steps below organize the material into a workflow.

1. **Specify the result schema.** Student IDs, names, course titles, or one statistics tuple per department?
2. **Identify objects and the candidate universe.** Use keys for namesakes; "never" requires the complete candidate population.
3. **List complete join conditions.** Check keys and context such as term; do not natural-join merely because names match.
4. **Translate quantifiers and logical words.** At least one usually uses joins/projection; or uses union; both uses intersection; never subtracts existing matches from all candidates; every uses division or removal of candidates with missing pairs.
5. **Keep attributes needed later.** Filter, join, and make identity-based set decisions before projecting presentation attributes.
6. **Track intermediate schemas.** Are selection attributes available? Are difference inputs compatible? Does division return the intended candidate attributes?
7. **Try small counterexamples.** Shared names, several matches, no matches, extra courses, one missing target, and an empty target.

The six Tutorial 2(a) categories correspond to selection/projection, cross-relation joins, union/intersection, difference, division, and γ. These notes teach the concepts and methods; they do not reproduce a complete tutorial answer set or create the later English practice papers.

14. Common distinctions, revision checklist, and short-answer language

Pair	Distinction
σ / π	Filters rows and preserves schema / retains columns and deduplicates
Undefined expression / empty result	Referring to an unavailable attribute is an error / a valid expression can have no matches
Union-compatible / equal column count	Aligned attributes, names, and domains under the lecture convention / column count alone is insufficient
Or / exactly one category	Includes both / must exclude both

Pair	Distinction
Equi-join / natural join	Explicit equality with attribute copies retained / equality on all common names with columns merged
Student name / identifier	Names can coincide / keys track the same object
Some record does not match / never matches	Existence of a nonmatching record / absence of every matching record
Every / only	Covers the required targets / excludes objects outside the target set
Division targets / candidates	All tuples of S / usually values from $\pi_X(R)$
Counting join rows / counting distinct courses	Pairing count / requires retaining course identity and deduplicating appropriately

These are learning goals, not a formal practice test or an official mark allocation:

- ☐ State input requirements, result schemas, and duplicate behaviour for every operator.
- ☐ Work out selections, projections, ID set operations, and natural joins on the example.
- ☐ Use renaming correctly for two roles of one relation.
- ☐ Explain premature name projection with the namesake counterexample.
- ☐ Explain why tutorial joins require checking term.
- ☐ Identify division's X, Y, and complete target relation and calculate the quotient.
- ☐ Expand division using differences of missing required pairings.
- ☐ Distinguish standard division from an explicit-universe version for empty targets.
- ☐ Calculate grouped statistics and detect join multiplication or absent zero-count groups.
- ☐ State the conditions required for each equivalent rewrite.

Useful short-answer formulations:

> Selection filters tuples while preserving the relation's attributes.
>
> Projection retains the specified attributes and eliminates duplicate tuples.
>
> Union, intersection, and difference require union-compatible relations.
>
> A natural join matches all common attributes and retains one copy of each common attribute.
>
> Perform identity-based set operations before projecting names, because different entities may share a name.
>
> Division returns candidates associated with every tuple in the divisor.
>
> Universal conditions can be expressed by removing candidates that have a missing required pairing.
>
> The object counted depends on the input tuples and the identifiers retained before aggregation.

Sources and preparation record

- Main lecture deck: Relational_Algebra.pptx, text of slides 1–61 read. Original diagrams/formulas on slides 7, 10, 17, 19, 37–38, 40, 42, 48, 50, 53–54, 57, and 60–61 were visually inspected for selection/projection conditions, union compatibility, joins, division, and basic-operator notation.
- Scope: CSC5041_Teaching_Plan.pdf, page 1, Week 4, and page 2, Tutorial 2. Supporting questions: Tutorial 2.pdf, page 1, Question 1(a)(i)–(vi); the original page and underlined keys were inspected. The plan labels Tutorial 2 Week 5, while the question sheet labels it Week 6. This chapter follows the actual topics without inferring an examination rescheduling from that discrepancy.
- The simplified schema, namesake counterexample, empty-target/candidate-universe discussion, aggregation notation, and counting boundaries are distinguished from verbatim lecture statements. Full SQL NULL and duplicate semantics are outside this chapter.
- LaTeX edition prepared on 20 October 2026; original evidence status is retained. No new database execution is claimed.
- Input SHA-256: algebra deck `dcacfada73c2f482ab3b83ac7c8312cb2176932906b6d3e6cac071d01932283d`; Tutorial 2 `72c016526c1c0ff02c10fcb5f7f964c1184999c684228671025800936a6cabac`; syllabus teaching plan `31380a7fe6504e46541b6db0bb2f24859c69188bfeb9681dca73354ebeb68295`.

06 SQL Queries: English Study Notes

CSC5041 · In-class Test 1 revision · 19 October 2026

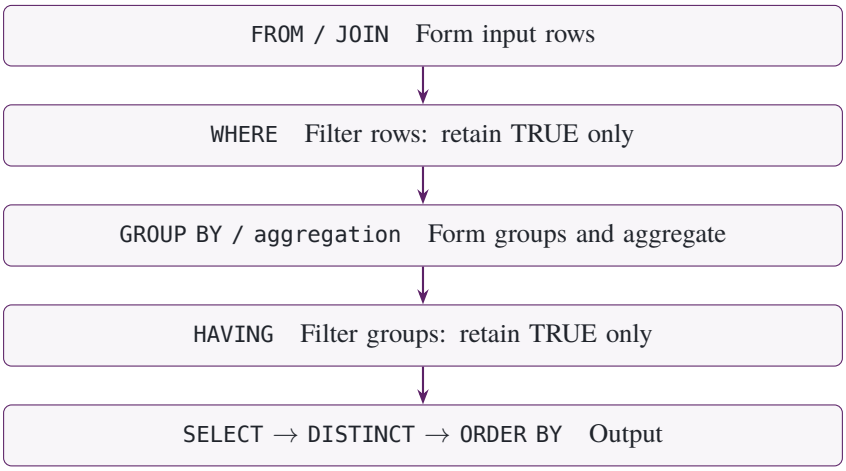
Scope and learning goals

This chapter follows 05 Relational Algebra. Its main sources are the lecturer's SQLpart1.pptx, slides 3–58, and the query material on slides 2–24 of SQLpart2.pptx. P1 and P2 below distinguish the decks; numbers refer to slide order.

Week 5 on page 1 of the teaching plan includes SELECT–FROM–WHERE, sorting, aggregation, grouping, and nested queries. Tutorial 2 on page 2 explicitly lists GROUP BY, HAVING, NOT EXISTS, and NOT IN with NULLs. Stopping at the file named Part 1 would therefore omit listed topics. Advanced quantified comparisons, DDL/DML, views, and PL/pgSQL are distinguished rather than treating all Part 2 material as compulsory for Test 1.

After studying this chapter, you should be able to choose a result granularity, specify complete joins, and handle duplicates, NULL, grouping, existence, and nonexistence. All data is original. **Query results are static deductions; no queries were executed in PostgreSQL or another database.** Supplementary official documentation checks semantics, not the lecturer's examination depth.

1. Six common clauses: written and logical order



Logical stages for ordinary queries, not a mandatory physical execution order.
WHERE cannot directly use same-level AVG(score); HAVING can filter aggregate results.

Source: P1 slides 13, 26–27, and 44–45; P2 slides 18 and 22–24.

```
SELECT output columns or expressions
FROM input tables and joins
WHERE row conditions
GROUP BY grouping attributes
HAVING group conditions
ORDER BY ordering attributes;
```

Use WHERE, GROUP BY, HAVING, and ORDER BY as needed. P2 slide 24 presents the course frame-

work, not an exhaustive upper limit on modern SQL syntax.

For ordinary queries, use these **logical stages**:

```
FROM / JOIN → WHERE → GROUP BY / aggregation → HAVING
            → SELECT → DISTINCT (if specified) → ORDER BY
```

This is not a mandatory physical execution order. WHERE filters before groups form, so `AVG(score)` cannot directly be a same-level row condition. Do not use an output alias in the same-level WHERE; ORDER BY can use output aliases.

SQL querying is generally declarative: specify the desired result. A computed SELECT expression changes the output, not the stored values as UPDATE would.

2. Running example: add terms and missing scores to Chapter 05

These are original inputs, **not the tutorial schema**, and the simplified Takes relation from Chapter 05 is not reused as a full enrolment table.

```
Student(sid, sname, major)                PK{sid}
Course(cid, title, dept, credits)         PK{cid}
Enrolment(sid, cid, term, score)          PK{sid,cid,term}
Enrolment.sid → Student.sid; Enrolment.cid → Course.cid
```

All columns except score are mandatory. score is nullable numeric data; NULL means a score is not yet available, not zero. term is a string.

Student:

sid	sname	major
S1	Amy	CS
S2	Ben	Math
S3	Amy	CS
S4	Dana	Math

Course:

cid	title	dept	credits
C1	Databases	CS	4
C2	AI	CS	3
C3	Linear Algebra	Math	4

Enrolment:

sid	cid	term	score
S1	C1	2026F	90
S1	C2	2026F	80
S1	C1	2027S	NULL
S2	C1	2026F	NULL
S2	C3	2026F	80
S3	C3	2026F	NULL

There are six enrolments, three non-NULL scores, and two distinct non-NULL scores: 80 and 90. S1 repeats C1, S1 and S3 share a name, and S4 has no enrolments. Unless an explicit hypothetical change is stated, each example independently uses these inputs.

3. Basic queries, aliases, strings, and DISTINCT

Source: P1 slides 12–13, 18–25, and 28–40.

```sql
SELECT c.cid, c.title, c.credits * 2 AS doubled_credits
FROM Course AS c
WHERE c.dept = 'CS' AND c.credits > 3;
```

The result is `(C1,Databases,8)`; the stored credits value remains 4. Use straight single quotes for strings and double an embedded quote, as in `'O''Brien'`. Do not paste typographic quotation marks from slides into executable SQL.

Expression	Meaning
=, <>, <, <=, >, >=	Comparisons; these notes use <> for inequality
`IN ('CS','Math')`	Membership in a value list
`LIKE 'A%'`	Starts with A; % matches zero or more characters
`LIKE '_e%'`	Has e as its second character; _ matches one character
`BETWEEN 3 AND 4`	Inclusive range comparison

BETWEEN is supplementary syntax. Parenthesize combined AND/OR conditions explicitly, for example `(major='CS' OR major='Math') AND sid<>'S4'`.

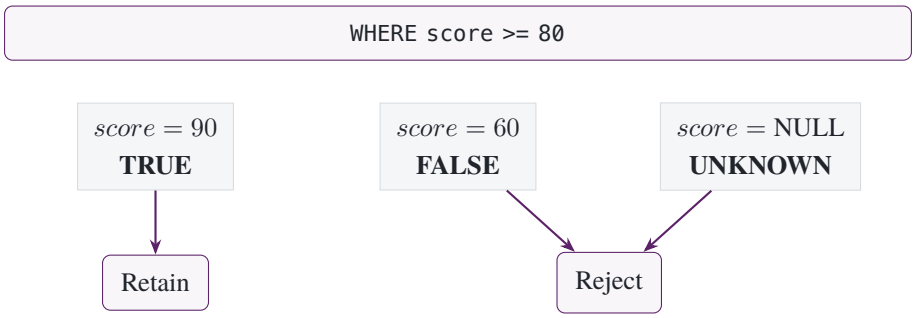
SQL retains output duplicates by default

```sql
SELECT s.sname FROM Student AS s;
SELECT DISTINCT s.sname FROM Student AS s;
```

The first outputs Amy twice, Ben once, and Dana once. The second returns three distinct names. No row order is guaranteed without ORDER BY. `DISTINCT sid,sname` deduplicates the **complete output tuple**, not each column independently.

Requesting names does not make names identifiers. Prefer sid for joins and identity-based set decisions.

4. NULL and three-valued logic



FALSE and UNKNOWN differ logically, but WHERE retains neither.
Use IS NULL to test for NULL, not = NULL. Numbers are original teaching examples.

Source: P1 slides 18 and 35–36; additional check: PostgreSQL comparison documentation.

Ordinary comparisons with NULL generally produce **UNKNOWN**. WHERE retains only TRUE rows. FALSE and UNKNOWN are both rejected, but they are different logical results.

Expression	Result
`NULL = NULL, NULL <> NULL, NULL > 80`	UNKNOWN
`NOT UNKNOWN`	UNKNOWN
`TRUE AND UNKNOWN`	UNKNOWN
`FALSE AND UNKNOWN`	FALSE
`TRUE OR UNKNOWN`	TRUE
`FALSE OR UNKNOWN`	UNKNOWN

Slide clarification: do not memorize "Comparison against NULL yields FALSE" on P1 slide 18 as a logical law. Slide 35 explicitly gives UNKNOWN. Rejection by WHERE does not make the expression itself FALSE.

```sql
SELECT sid, cid, term
FROM Enrolment
WHERE score IS NULL;
```

The result contains `(S1,C1,2027S)`, `(S2,C1,2026F)`, and `(S3,C3,2026F)`, without a guaranteed order. `score = NULL` is not a replacement. Nor does `score <> 90` retrieve missing scores: those comparisons are UNKNOWN.

To include scores other than 90 together with missing scores, write `score <> 90 OR score IS NULL`. Zero is a known numeric score; replacing missing values with zero requires a substantive justification.

5. Multiple-table joins: identify the full association

Source: P1 slides 41–49 and 52–58.

```sql
SELECT s.sid, s.sname, c.title, e.term
FROM Student AS s
JOIN Enrolment AS e ON e.sid = s.sid
JOIN Course AS c ON c.cid = e.cid
WHERE c.dept = 'CS';
```

There are four rows: S1's two C1 terms and one C2 term, plus S2's C1 term. Selecting only names would produce Amy three times and Ben once. If distinct students are required, use `DISTINCT s.sid,s.sname` rather than masking an incorrect join.

Missing join conditions can create a Cartesian product. With two name columns, `name=name` can also be ambiguous. Qualify columns with aliases to make every association explicit.

NATURAL JOIN risks

It matches **all common column names** and merges those columns. P1 slides 55–57 explain that instructor and course departments need not agree. Prefer clear JOIN…ON conditions for the business association actually required.

Tutorial 2's Enrolled and Teaches contain both cid and term. Relating teaching assignments to enrolments requires `e.cid=t.cid AND e.term=t.term`. Joining only cid mixes offerings from different terms. This chapter has no Teaching table, so that condition is not presented as a complete runnable query on these inputs.

Some INNER JOIN conditions can move between ON and WHERE without changing the result. This does not dictate physical execution order and does not generalize to all outer joins, which introduce NULL-extended rows.

6. Self-joins and ordering

Source: P1 slides 50–51 and P2 slide 23.

Pairs of different students sharing a major:

```sql
SELECT a.sid AS sid1, b.sid AS sid2
FROM Student AS a
JOIN Student AS b ON a.major = b.major
WHERE a.sid < b.sid
ORDER BY sid1, sid2;
```

Under the example's ordinary character ordering of IDs, the result is `(S1,S3)`, then `(S2,S4)`. `<` excludes self-pairs and reverse duplicates; `<>` alone retains both directions.

```sql
SELECT cid, title, credits
FROM Course
ORDER BY credits DESC, cid ASC;
```

The course order is C1, C3, C2. Ordering only by credits leaves the tie between C1 and C3 unresolved. Add an explicit secondary key when a definite order is required. Ordering does not eliminate duplicates or change tuple identity.

7. Aggregates: COUNT, SUM, AVG, and NULL

Source: P2 slides 12–16; additional check: PostgreSQL aggregate documentation.

```sql
SELECT COUNT(*) AS n_rows,
       COUNT(score) AS n_scored,
       COUNT(DISTINCT score) AS n_distinct_scores,
       SUM(score) AS total_score,
       AVG(score) AS mean_score
FROM Enrolment;
```

The result is `(6,3,2,250,250/3)`, with a mean of approximately 83.33; displayed precision depends on the numeric type.

Expression	What is counted or how NULL is handled
`COUNT(*)`	All input rows, including rows with missing score
`COUNT(score)`	Non-NULL score occurrences, including repeated values
`COUNT(DISTINCT score)`	Distinct non-NULL score values
`SUM/AVG/MIN/MAX(score)`	Calculated from non-NULL input values

AVG divides by three non-NULL scores, not six enrolments. `AVG(DISTINCT score)` instead gives `(80+90)/2=85`, changing the statistical question rather than providing a general duplicate repair.

Empty input and all-NULL input

Without GROUP BY, an aggregate query still returns one row for no matching input: COUNT is zero, while SUM, AVG, MIN, and MAX are NULL. If matching rows exist but every score is NULL, COUNT(*) counts those rows, COUNT(score)=0, and the other four score aggregates remain NULL.

Filtering to `sid='S3'`, for example, gives COUNT(*)=1, COUNT(score)=0, and AVG(score)=NULL. Missing summaries are not automatically zero; choosing a zero display requires appropriate business meaning.

8. GROUP BY and HAVING: one row per group

Source: P2 slides 17–22.

```sql
SELECT e.sid, COUNT(*) AS n_enrolments,
       COUNT(e.score) AS n_scored, AVG(e.score) AS mean_score
FROM Enrolment AS e
GROUP BY e.sid
HAVING COUNT(*) >= 2
ORDER BY e.sid;
```

sid	n_enrolments	n_scored	mean_score
S1	3	2	85
S2	2	1	80

WHERE filters input rows; HAVING filters formed groups. Adding `WHERE score IS NOT NULL` changes

COUNT(*) to a count of scored enrolments. S2 then fails the two-row requirement, so this is not equivalent to the original query.

Use the course's explicit writing rule: include nonaggregated SELECT columns in GROUP BY. Do not select `sid,cid,AVG(score)` while grouping only by sid and arbitrarily choose a course for each student. Some implementations recognize functional-dependency exceptions; writing the grouping explicitly avoids relying on them here.

Counted objects and zero enrolments

S1 has three enrolments but only two different courses; use `COUNT(DISTINCT cid)` for course count. Joining Enrolment can also repeat a course across students. Decide whether the query counts enrolment records or distinct courses first.

The query does not generate a zero-count group for S4. Covering all students, including zero enrolments, needs another query design, usually involving an outer join. Do not claim that the current inner-join/grouping result includes S4.

9. Subqueries: scalar versus multiple rows

Source: P2 slides 2–5; cardinality boundaries additionally checked against PostgreSQL subquery documentation.

Scalar subquery

```sql
SELECT cid, title
FROM Course
WHERE credits > (SELECT AVG(credits) FROM Course);
```

The mean credits value is 11/3, so C1 and C3 qualify. The inner aggregate returns one column and one row, making it suitable as one value.

A scalar position requires one column and at most one row. Zero rows mean NULL; multiple rows are an error under PostgreSQL/standard scalar semantics, not an instruction to take the first row. Replacing the inner query with `SELECT credits FROM Course WHERE dept='CS'` returns two rows and is invalid as a scalar. The inner AVG does not need an outer alias reference.

IN: value membership

```sql
SELECT s.sid, s.sname
FROM Student AS s
WHERE s.sid IN (
    SELECT e.sid
    FROM Enrolment AS e
    WHERE e.score >= 85
);
```

Only S1 qualifies. A scalar IN subquery returns one column. Repeated right-side values do not multiply an outer row. Section 11 discusses NULL effects.

10. EXISTS, NOT EXISTS, and correlation

Source: P2 slide 6; the Tutorial 2 topic list includes NOT EXISTS.

A correlated subquery references the current outer row, as in `e.sid=s.sid`. Interpret it as asking an existence question for each student; this does not require the engine to execute the complete inner query once per row.

```sql
SELECT s.sid, s.sname
FROM Student AS s
WHERE EXISTS (
    SELECT 1
    FROM Enrolment AS e
    WHERE e.sid = s.sid AND e.score IS NULL
);
```

S1, S2, and S3 qualify. EXISTS tests for rows, not the selected constant 1. In this ordinary existence query, multiple inner matches still produce one result per outer student. EXISTS does not independently remove duplicates already present in the outer input.

```sql
SELECT s.sid, s.sname
FROM Student AS s
WHERE NOT EXISTS (
    SELECT 1
    FROM Enrolment AS e
    WHERE e.sid = s.sid
);
```

Only S4 qualifies. "Never" means **no matching row**, not one row with `cid <> 'C1'`. Omitting the correlation can make any enrolment in the entire table determine the condition for all students.

11. NOT IN with NULL: explain the counterexample

Source: Tutorial 2 on page 2 of the plan; additional reference: PostgreSQL IN/NOT IN.

For non-NULL numbers, `80 NOT IN (90,NULL)` is `80<>90 AND 80<>NULL`, yielding UNKNOWN and rejection by WHERE. `90 NOT IN (90,NULL)` is FALSE because an equal value exists. **A NULL on the right does not force every NOT IN result to be identical.**

Situation	Scalar membership result
An equal right-side value exists	IN is TRUE; NOT IN is FALSE
No equal value, NULL participates, and the right side is nonempty	Can be UNKNOWN rather than definite nonmembership
Right side empty	IN is FALSE; NOT IN is TRUE

This particular query for unenrolled students is safe under the running example:

```sql
SELECT s.sid, s.sname
FROM Student AS s
WHERE s.sid NOT IN (SELECT e.sid FROM Enrolment AS e);
```

Both sid columns are guaranteed non-NULL, so S4 is returned. Replacing them with nullable columns

invalidates that reasoning. Filtering NULL out of the right side does not automatically settle a NULL left side's meaning. Correlated NOT EXISTS also requires an explicit interpretation of NULL-key matching; it is not unconditionally equivalent to NOT IN.

12. UNION, INTERSECT, EXCEPT, and identity

Source: P2 slide 9; corresponds to set operations in Chapter 05.

```sql
SELECT e.sid
FROM Enrolment AS e
JOIN Course AS c ON c.cid = e.cid
WHERE c.dept = 'CS'
INTERSECT
SELECT e.sid
FROM Enrolment AS e
JOIN Course AS c ON c.cid = e.cid
WHERE c.dept = 'Math';
```

The result is S2. The two department conditions need not hold for one enrolment row: each side finds students separately, then intersects their identities.

Operation	Default behaviour
UNION	Combine and remove duplicate output tuples
UNION ALL	Combine and retain multiplicities
INTERSECT	Common tuples, deduplicated
EXCEPT	Left tuples absent from the right, deduplicated; direction matters

Inputs require equal column counts and compatible types at corresponding positions. SQL set operations align columns positionally; unlike the course's named relational-algebra convention, equal column names are not required. INTERSECT ALL and EXCEPT ALL have multiplicity semantics; distinguish them from the default deduplicated forms rather than treating ALL as decorative.

Projecting sname on each side first would combine S1's CS enrolments with S3's Math enrolment through the shared name Amy. Work with sid first, then join or use IN/EXISTS to obtain names.

13. Every target: no required match is missing

Source: P2 slides 10–11. This original example translates Chapter 05's method into SQL; it is not a tutorial answer sheet.

Start with Student candidates and exclude anyone for whom a CS course exists without a corresponding enrolment:

```sql
SELECT s.sid, s.sname
FROM Student AS s
WHERE NOT EXISTS (
    SELECT 1
    FROM Course AS c
    WHERE c.dept = 'CS'
      AND NOT EXISTS (
          SELECT 1
          FROM Enrolment AS e
          WHERE e.sid = s.sid AND e.cid = c.cid
      )
);
```

Only S1 qualifies. The innermost query looks for a student–course match; the middle query looks for a target with a missing match; the outer query requires no such target. This example explicitly allows enrolment in any term. If a specific term is required, state the corresponding target and enrolment scope.

Every target must be covered, but extra non-CS courses are allowed. Repeating one course cannot compensate for a missing course. Derive targets from complete Course, not only courses someone has already taken.

If hypothetically no CS courses exist, this returns all Students, including S4: no target can be missing. Add a nonempty-target requirement if the question demands one. P2 slide 11 enumerates candidates from Sells and therefore returns only bars occurring in Sells when its target set is empty. That differs from starting with all Student candidates here; connect this to Chapter 05's empty-divisor discussion.

14. Remaining slide material and scope

ANY / ALL [recognition supplement]

Source: P2 slides 7–8. Week 6 lists advanced subqueries and set comparison. This section only identifies the notation rather than imposing extra Test 1 depth.

`x > ANY (subquery)` asks whether at least one comparison is TRUE; `x > ALL (subquery)` asks whether all are TRUE. For an empty right side, ANY is FALSE and ALL is TRUE. NULL may produce UNKNOWN. Replacing ALL with a MAX comparison is not unconditional because empty or NULL input can change the meaning.

DDL, data modification, views, and PL/pgSQL

P2 slides 25–52 also cover CREATE TABLE, keys, INSERT, DELETE, UPDATE, views, and schema changes. Connect these to Chapters 03–04 without assuming that every topic in that file is a Test 1 query priority:

- PRIMARY KEY combines uniqueness and non-nullability. FOREIGN KEY does not automatically require non-NULL values, and one ordinary foreign key alone does not preserve 1:1.
- DELETE removes rows; DROP TABLE removes the table structure. UPDATE without WHERE affects the whole table. These commands are explained, not executed.
- An ordinary view is an interface defined by a query. Do not read "does not take up space" as claiming that even its definition metadata occupies no storage; it also differs from a materialized view storing results.
- PL/pgSQL, indexes, transactions, and normalization are outside this chapter.

15. Query-writing checks and revision checklist

1. **Specify result granularity.** One row per student, enrolment, distinct course, or group?
2. **Check schemas and complete join keys.** Pay attention to sid, cid, and term; distinguish student majors from course departments.
3. **Write row predicates and handle NULL.** WHERE retains only TRUE; choose IS NULL, IN, or EXISTS appropriately.
4. **Decide duplicate requirements.** Is DISTINCT appropriate to the result, or masking a faulty join?
5. **Group before filtering groups.** Explicitly group ordinary selected columns and distinguish COUNT(*), COUNT(column), and COUNT(DISTINCT column).
6. **Check subquery shape and correlation.** Is a scalar result at most one row? Does existence concern the current outer entity?
7. **Check boundaries.** Namesakes, repeated enrolments, missing scores, no matches, multiple matches, and empty targets; add ORDER BY when order matters.

Common confusion	Distinction
UNKNOWN / FALSE	Different logical results; WHERE retains neither
DISTINCT / GROUP BY	Output deduplication / forming statistical groups; not mechanical substitutes
WHERE / HAVING	Filter rows before grouping / filter groups after aggregation
COUNT(*) / COUNT(score)	All input rows / non-NULL score rows
EXISTS / scalar subquery	Any rows? / treat the result as one value
NOT IN / NOT EXISTS	Compare a value set / test correlated matches; not unconditionally equivalent with NULL
Every / only	Cover all required targets / exclude associations outside the target
SQL set operation / ordinary SELECT	Deduplicates by default / retains duplicates by default

These are learning goals, not a formal practice paper or an official mark allocation:

- ☐ State the written clause order and logical query stages.
- ☐ Work out basic queries, DISTINCT, self-joins, and ordering with secondary keys.
- ☐ Explain with three-valued logic why missing scores do not satisfy `score<>90`.
- ☐ Check complete conditions for ordinary and cross-term joins.
- ☐ Calculate the three COUNT variants and all-NULL/empty-input aggregates.
- ☐ Explain how WHERE and HAVING change the statistical result.
- ☐ Distinguish scalar, IN, and EXISTS subquery requirements.
- ☐ Explain the NULL counterexample for NOT IN and the safe non-NULL sid case.
- ☐ Perform identity-based set operations before projecting names.
- ☐ Explain all three levels of double NOT EXISTS and its empty-target behaviour.

Useful short-answer formulations:

> WHERE retains only rows for which its condition evaluates to TRUE.

> COUNT(*) counts rows, whereas COUNT(column) counts non-NULL values in that column.

WHERE filters input rows; HAVING filters groups after aggregation.

EXISTS tests whether a subquery returns any rows, not the values in its SELECT list.

A NOT IN comparison can evaluate to UNKNOWN when NULL participates and no equal value is found.

A universal requirement can be expressed as the absence of a required target without a matching association.

Sources and preparation record

- Lecturer's decks: SQLpart1.pptx and SQLpart2.pptx. Core sources are P1 slides 3–58 and P2 slides 2–24. Text was read and original slides P1 18, 31, 35–36, 51, 55, 58 and P2 2, 6, 9, 11, 15–16, 20, 22, 24 were visually inspected. P2 slides 25–52 are used only for scope recognition.
- Teaching scope: CSC5041_Teaching_Plan.pdf, page 1, Weeks 5–6, and page 2, Tutorial 2. Supporting questions: Tutorial 2.pdf, page 1. Chapter 05 records the sheet's Week 6 versus plan's Week 5 discrepancy; this chapter infers no rescheduling.
- Additional checks: PostgreSQL 18 official documentation on comparisons, subqueries, aggregates, and table expressions, read online this turn for NULL, subquery cardinality, aggregation, and related boundaries. This does not imply that the lecturer requires PostgreSQL 18.
- LaTeX edition prepared on 20 October 2026; original evidence status is retained. No new database execution is claimed.
- Input SHA-256: SQLpart1 `aa28c989fb3674e780418c78c483bba20ff5a50b7da067b0341e60410a68c8ed`; SQLpart2 `acd25cbd4bbd246f8901ada4c437c3f2d00873385961475cd4c941e9089bf45c`; Tutorial 2 `72c016526c1c0ff02c10fcb5f7f964c1184999c684228671025800936a6cabac`; syllabus teaching plan `31380a7fe6504e46541b6db0bb2f24859c69188bfeb9681dca73354ebeb68295`.